Lava — Bug Fix Audit Trail

> **Revision (2026-07-04, goapi archiveorg /http-download 502 → root-cause fix):** `/v1/archiveorg/http-download/{id}` returned HTTP 502 with body `{}` for a valid bare identifier. Root cause: the route uses a Gin wildcard path parameter (`*id`), so `c.Param("id")` returned `"/vidmate_201910_201910"` with a leading slash. `archiveorg.ProviderAdapter.resolveDownloadTarget` split on `/`, saw an empty first component, and returned `provider.ErrUnknown`, which `writeProviderError` mapped to 502. The same bug would have hit composite `identifier/filename` ids. Affected files: `lava-api-go/internal/handlers/v1/torrent.go` (`GetDownload` and `GetTorrent` id extraction), `lava-api-go/internal/handlers/v1/http_download_test.go`. Fix: `id := strings.TrimPrefix(c.Param("id"), "/")` in both handlers, plus regression test `TestHTTPDownload_TrimsLeadingSlashFromWildcardID` covering bare and composite ids. Verification (reproduce-first §6.T.1): before the fix the real endpoint returned `502 {}`; after the fix it returns `200` with the 2045-byte `.torrent` file. Falsifiability rehearsal: mutation `strings.TrimPrefix(c.Param("id"), "")` (no-op trim) makes the new test RED with `download id received by provider was "/vidmate_201910_201910", want "vidmate_201910_201910"`, then reverted and GREEN. Package tests: `go test ./internal/handlers/v1/ ./internal/archiveorg/` PASS. Container rebuilt (`podman-compose --profile api-go build --no-cache lava-api-go`) and verified on `https://127.0.0.1:8443`.

> **Revision §11.4.44 (2026-07-03, §6.AC archiveorg mapError hardening — egress-stall-masquerade class CLOSED):** the 2026-07-02 diagnostic trap is fixed reproduce-first. Root cause: `lava-api-go/internal/archiveorg/provider.go mapError()` collapsed EVERY error to a bare `provider.ErrUnknown`, discarding the real cause. When the containerized goapi's outbound TCP to IPv4-only `archive.org` stalled over the Mullvad VPN (a dial/TLS timeout), `writeProviderError`'s §6.AC default branch recorded a `RecordNonFatal` whose `error_message` was the useless generic `"provider: unknown error"` — so the egress stall MASQUERADED as an archiveorg parser bug and cost real diagnostic time. Fix: `mapError` returns `fmt.Errorf("%w: %v", provider.ErrUnknown, err)` — `errors.Is(_, provider.ErrUnknown)` stays true (HTTP status 502 + the recorded non-fatal are UNCHANGED, wire body stays `{}`)

while the underlying dial/TLS detail SURVIVES for remote triage. archiveorg was the ONLY discard-everything adapter — nnmclub/kinozal/rutracker already preserve the cause via `default: return err`, so they were left untouched. Verification (reproduce-first §6.T.1): `internal/archiveorg/provider_test.go TestMapError_PreservesUnderlyingCause` — RED `mapError discarded the underlying cause: got "provider: unknown error", want it to contain "i/o timeout"` before the fix, GREEN after (full archiveorg pkg 15/15 + `TestMapError_NilPassthrough` guard); v1 handler error-mapping + non-fatal-telemetry tests green (no status/wire regression); `go vet` clean. Observability-only change — the banked device-green goapi matrix (rutracker 1080p+mp3, kinozal 1080p, gutenberg sherlock) is unaffected. Public-provider keystones remain gated on the per-destination excluded-proxy egress decision (operator-present required).

Revision §11.4.44 (2026-07-02, goapi keystone GREEN): the goapi × rutracker autonomous-QA keystone (`Challenge70AutonomousQaProviderMatrixTest`) went from an honest §6.J SKIP to a device-verified `verdict=PASS` (real rutracker login + search + download, recorded to `qa_rec_0.mp4; .lava-ci-evidence/autonomous-qa/2026-07-02/goapi/rutracker-1080p/`). Five reproduce-first + Bluff-Audited fixes — three are REAL PRODUCTION BUGS a goapi-catalogue user would hit, all invisible to the prior green suite:

- **Fix B** — `lava-api-go/internal/rutracker/provider.go:48 DisplayName() "RuTracker"` → `"RuTracker.org"`. Root cause: the name equalled the id case-insensitively, so the Android catalogue humanizer (`core/data/.../ProviderCatalogRepository.friendlyDisplayName`) rendered the off-brand `"Rutracker"`, diverging from the bundled `"RuTracker.org"` and breaking Challenge70's provider pick. Verification: `provider_mapping_test.go TestProviderAdapter_Metadata` (mutation → `DisplayName = "RuTracker", want RuTracker.org`).
- **Fix C2** — `core/data/.../ProviderCatalogRepository.toRemoteDescriptor` now derives `AUTH_REQUIRED` when `authType != NONE` (`withAuthRequiredWhenAuthTyped`). Root cause: the goapi catalogue declares `authType:CAPTCHA_LOGIN` but omits an `AUTH_REQUIRED` capability, so `ApiBackedTrackerClient.getFeature<AuthenticatableTracker>()` (gated on it) returned null → `sdk.login` short-circuited to null → no session cookie (§6.E capability honesty). Verification: `ProviderDisplayNameTest` (mutation → `AssertionError at ProviderDisplayNameTest.kt:133`).
- **Fix D** — `core/tracker/client/.../ApiBackedTrackerClient` blocking OkHttp `.execute()` (getString/getBytes/postJson/

healthCheck) now runs on an injected `Dispatchers.IO`. Root cause: `sdk.login` runs on `viewModelScope` (Main) → the un-dispatched blocking call threw `android.os.NetworkOnMainThreadException` on every goapi login/search/download. Falsifiability: device keystone `NetworkOnMainThreadException` before, real 2.5s network login after.

- **Fix E** — client `LoginResultDto` now mirrors the real `provider.LoginResult {success, authToken, expiresAt}` wire. Root cause: the prior `{state, sessionToken}` DTO required a `state` field the server never sends → `kotlinx.serialization.MissingFieldException` → a genuinely-successful login surfaced as "Connection test failed", no cookie stored. Verification: `ApiBackedTrackerClientLoginContractTest` (new real-stack, mutation → `MissingFieldException at ...kt:98`) + `ApiBackedTrackerClientTest` (corrected a bluff mock that used the fake `{state,…}` body — it passed green while the real login was broken). Contract note: the OpenAPI `/login` `AuthResponseDto {type,user}` schema is stale vs BOTH server and client — a tracked follow-up (reconcile openapi + server + client onto one shape).
- **Problem B (test-harness, not a product bug)** — `Challenge70` deselects the full provider list via `SelectAllProvidersTestTag` then re-selects requested providers via `performScrollTo`. Root cause: all 13 catalogue providers are selected by default; the prior exact/case-sensitive/no-scroll name-list deselect cleared only 5, stranding the wizard on a non-requested Configure page → "All set!" never rendered → SKIP.

Revision §11.4.44 (2026-06-24, updated again): appended three Crashlytics-triage fixes from 2026-06-24 session — ALL THREE FIXES LANDED in the working tree (the earlier "P1 OWED" note was a stale-`git diff` race artifact, corrected here): P0 CredentialsKeyHolder locked FATAL (`58a1335272bc`, fix in `CredentialsEntryRepositoryImpl.observe()` runCatching, §6.O closure log), P1 LazyColumn 2nd-site nested-scroll FATAL (`c7c8cccad09f`, fix `feature/search_input/.../SearchInputScreen.kt:96` `modifier = Modifier.weight(1f)` + §6.Q scanner CHECK 2 — scanner FAIL→PASS, "no nested-scroll antipattern detected"), P2 TopicPageDto MissingFieldException NON_FATAL (`8cde0ac208b3`, **PARTIAL** — `commentsPage` default only; the real IA crawl-topic payload omits all 6 fields so the full fix is a tracked follow-up, §6.O closure log). Source: `docs/issues/2026-06-24-crashlytics-full-triage.md`. All three are reproduce-first + Bluff-Audit'd and ship in 1.3.11-1073.

> **Revision §11.4.44 (2026-06-24, updated):** appended search cancel/timeout fix covering all 3 root causes — `SearchResultViewModel` back-press cancellation + 25 s client-side `withTimeout` (Bug 1+2, commit `20d98914`) + engine 18 s total-search deadline in `handlers/v1/search.go` + stale YTS mirror refresh (Bug 3, commits `0e81730b` + `1aa42536` + `3c1fa159`). 3 client regression tests + 2 engine Go tests added; all falsifiable. Final SHAs confirmed from `git show --stat`.

> **Revision §11.4.44 (2026-06-23):** appended five fixes from the 2026-06-23 session — H1 search-401 (AuthInterceptor handoff-key overwrite), api-app-17 stale-binary (§6.Z wrong-binary saga), ApiHttpException type regression, ApiKeyClientTest compile break, and firebase-distribute SIGPIPE. Each entry cites its real commit SHA verified against `git show --stat`.

> **Revision §11.4.44 (2026-06-08):** appended the 2026-06-08 session's six real defect fixes — the four §6.E capability-honesty bluffs (kinozal / nnmclub / rutor magnet wire-through + gutenberg `TORRENT_DOWNLOAD` drop), the HelixQA broken-pin go.mod conflict that broke `lava-api-go`'s build, and the nav-compose `2.9.0 → 2.9.1` test-teardown lifecycle race. Each entry below cites its real commit SHA + real files (verified against `git show --stat`). Per §11.4.6 no-guessing, details that could not be confirmed from git are marked `UNCONFIRMED`.

Per constitutional clause **§6.T.4 (Bugfix Documentation)** — every bug fix in this project MUST be documented here with root cause analysis, affected files, fix description, link to the verification test/ challenge, and the commit SHA that landed the fix.

§6.O (Crashlytics-Resolved Issue Coverage Mandate) extends this for Crashlytics-recorded issues; their closure logs live at `.lava-ci-evidence/crashlytics-resolved/<date>-<slug>.md`. §6.T.4 covers the rest — operator-reported, self-discovered, or reviewer-flagged bugs that don't enter the Crashlytics pipeline.

Format per entry:

```
## YYYY-MM-DD — <short slug>

**Root cause:** ...
**Affected files:** ...
**Fix:** ...
**Verification test/challenge:** path or commit ref
**Fix commit:** SHA
**Forensic anchor:** (optional) what surfaced the bug
```

2026-07-02 — AuthInterceptor decrypt crashed EVERY authenticated request on cert/blob drift (fail-open fix)

Discovered by: the goapi-keystone LAYER-1 root-cause (candidate 1) — the public `/providers` catalogue fetch failed at runtime, forcing the registry to fall back to bundled providers.

Root cause: `core/network/impl/.../AuthInterceptor.kt` `intercept()` HKDF-derived an AES key from the APK signing-cert sha256 and called `AesGcm.decrypt(blob,…)` inside `try { … } finally { … }` with **no catch**. On any signing-cert/blob drift (an androidTest build, a re-signed APK), `decrypt` throws `AEADBadTagException` which propagated out of `intercept()` → OkHttp wrapped it as `IOException` → EVERY request on the `@Named("lan")` client died, including the PUBLIC `/providers` fetch. A crypto failure on ONE optional header must not crash all traffic.

Fix (additive, fail-open): the auth-attach path (derive+decrypt+build) runs in a `try`; `chain.proceed(request)` moved OUTSIDE the try (genuine network errors never swallowed / double-proceeded). On failure: record a §6.AC non-fatal via the module's `LoggerFactory` (error CLASS only — never blob/key/nonce/header, §6.H) + send the request WITHOUT the `Lava-Auth` header (public endpoints don't need it; gated ones already handle the 401). `keyBytes.fill(0)`/`uuidBytes?.fill(0)` still run in `finally` (core auth-memory hygiene). `AuthInterceptorModule` threads the real `LoggerFactory`.

Test: `AuthInterceptorTest` — mismatched cert hash → decrypt throws; PRIMARY assertion on the wire: server RECEIVES `/providers` with NO auth header (not a crash); secondaries: a non-fatal naming `AEADBadTagException` is logged + does not contain the blob/nonce. RED (catch removed): the new test FAILS with `AEADBadTagException`; other 2 stay green. Reverted: yes. **Fix commit:** this commit.

2026-07-02 — NNM-Club responses decoded as UTF-8 despite windows-1251 (Cyrillic mojibake)

Discovered by: a Prowlarr Cardigann cross-check of the tracker parsers.

Root cause: NNM-Club (phpBB, windows-1251) responses were read via OkHttp `ResponseBody.string()` which defaults to UTF-8 when the HTTP `Content-Type` omits a charset (the live site declares

it only in a `<meta>` tag OkHttp ignores) → Cyrillic mojibake in search titles + broken Cyrillic login. Sibling **kinozal** already solves this with a win-1251 `bodyString()` helper.

Fix (additive, mirrors kinozal): `NnmclubHttpClient` gained `bodyString(response) = String(response.body?.bytes() ?: return "", windows-1251)`; `NnmclubSearch.kt:43` + `NnmclubAuth.kt:42` now use `http.bodyString(it)`.

Test: `NnmclubWindows1251DecodeTest` — MockWebServer serves a windows-1251 body with Cyrillic title `Дюна: Часть вторая (2024)` (both charset-header-absent — the live discriminator — and header-present); asserts the parsed `SearchResult` title equals the Cyrillic string byte-for-byte (§6.J). RED (decode reverted to UTF-8): `expected:<[Дюна: Часть вторая]…> but was:<[????: ?????? ??????]…>`. Reverted: yes. **Fix commit:** this commit. **Tracked follow-up (§6.AB, same class):** `NnmclubTopic.kt:41`, `NnmclubComments.kt:29`, `NnmclubBrowse.kt:34`, `NnmclubClient.kt:43` still use raw `it.body?.string()` and will get the same `http.bodyString(it)` swap in a follow-up.

2026-07-02 — LAYER 2: bundled RuTrackerClient search short-circuits Unauthorized (token-store mismatch)

Discovered by: the goapi-keystone LAYER 1/2 static+device trace (`/run/media/milosvasic/DATA4TB/.build-tmp/goapi-keystone-layers-2026-07-02.md`, LAYER 2). This is the "bundled-search-failure" layer the goapi CASE-COOKIE entry below explicitly deferred as a SEPARATE open investigation — now root-caused and fixed. After a genuinely-successful multi-tracker SDK login, the bundled `RuTrackerClient.search()` returned "problem reaching the trackers" with **no `tracker.php` request on the wire at all**.

Root cause (token-store mismatch):

- `RuTrackerSearch.search()` (`core/tracker/rutracker/.../feature/RuTrackerSearch.kt:31`) reads `token = tokenProvider.getToken()`, then `GetSearchPageUseCase` wraps `api.search()` inside `WithTokenVerificationUseCase`. That guard (`WithTokenVerificationUseCase.kt:12/15`) throws `Unauthorized` when `VerifyTokenUseCase(token)` (`= token.isNotEmpty()`) is false — i.e. the `tracker.php` request is *inside* the guarded block and never runs on an empty token.
- `tokenProvider` is `AuthServiceImpl` (bound in `core/auth/impl/.../di/AuthModule.kt`). `AuthServiceImpl.getToken()` reads `preferencesStorage.getAccount()?.token.orEmpty()`.
- The SDK login path `RuTrackerAuth.login → LoginUseCase` obtains the rutracker session cookie but NEVER wrote it to that account/token store — only the legacy single-tracker

`AuthServiceImpl.login()` did. So after an SDK-path login, `getAccount()` is null → `getToken()` returns `""` → the guard fires before any request. A genuinely-logged-in (cookie-valid) session was reported Unauthorized.

Fix (additive, §6.J — persist the real token, do NOT weaken the guard):

- `core/auth/api/.../TokenProvider.kt`: new `persistProviderToken(token)` seam (default no-op so the change is additive; production impls MUST override).
- `core/auth/impl/.../AuthServiceImpl.kt`: overrides `persistProviderToken` to write the session token into `preferencesStorage` (preserving any existing legacy account fields; token-only `Account` when none exists) so `getToken()` returns it. No auth-state emission (the SDK observation layer uses `signalAuthorized`).
- `core/tracker/rutracker/.../feature/RuTrackerAuth.kt`: after a successful login, bridges `result.sessionToken` into the store via `persistProviderToken(...)`.

Verification (reproduce-first, Bluff-Audited RED→GREEN): `core/tracker/rutracker/src/test/kotlin/lava/tracker/rutracker/feature/RuTrackerSearchAfterSdkLoginTest.kt` — real `RuTrackerAuth+LoginUseCase` (login) and real `RuTrackerSearch+GetSearchPageUseCase+WithTokenVerificationUseCase` (search) over a `MockWebServer`, sharing ONE in-memory `TokenProvider` (behaviorally equivalent to `AuthServiceImpl`'s token store). PRIMARY assertion on the returned `SearchResult` (parsed torrent id/title/seeders); secondary: exactly one `tracker.php?nm=` request issued. RED when the `RuTrackerAuth.login` bridge is removed (search throws `lava.tracker.rutracker.model.Unauthorized`, 0 tracker.php requests).

Fix commit: this commit.

2026-07-02 — goapi CASE-COOKIE: provider login session never reached the ApiBackedTrackerClient search (Auth-Token dropped)

Discovered by: the autonomous-QA rutracker/goapi keystone on a real containerized-KVM emulator — search failed with "problem reaching the trackers" even though onboarding + rutracker login succeeded (bb_session obtained).

Root cause (two additive gaps in the `ApiBackedTrackerClient` session-token path):

- **A — onboarding never stored the session token.** `ProviderSessionTokenHolder` was written ONLY by `ProviderLoginViewModel` (settings re-login); the ONBOARDING login path (`OnboardingViewModel`, the Authenticated branch) never called `ProviderSessionTokenHolder.set(...)`. So the per-provider `bb_session` captured during onboarding was dropped.
- **B — the token was read only at client BUILD time.** `ApiBackedTrackerClient .withAuth()` attached `Auth-Token` from the constructor-captured `sessionToken`, which the factory reads when the dynamic client is BUILT. Onboarding builds that client at the ApiSelection step — BEFORE the provider login stores the token — so even after fix A the built client still had `sessionToken=null` and `withAuth()` omitted `Auth-Token` → the Go API 401s `/v1/{provider}/search`.

Fix (additive, §6.J):

- A: `feature/onboarding/.../OnboardingViewModel.kt` Authenticated branch now calls `ProviderSessionTokenHolder.set(currentId, loginResult.sessionToken)` (mirrors `ProviderLoginViewModel.kt:388`).
- B: `core/tracker/client/.../ApiBackedTrackerClient.kt` `withAuth()` now reads `ProviderSessionTokenHolder.tokenFor(descriptor.trackerId)` LIVE at request time (fallback to the constructor param) — so a token stored after build is threaded, AND the in-memory-holder cold-restart gap is closed.

Verification (reproduce-first, both Bluff-Audited RED→GREEN):

- A: `OnboardingViewModelApiSelectionFlowTest.test and continue with valid credentials persists the provider session token` — asserts the holder gets the token; RED when the `.set(...)` line is removed.
- B: `ProviderSessionTokenEndToEndWiringTest .sessionStoredAfterClientBuild_isStillThreaded_liveAtRequestTime` — builds the client with the holder empty, stores the token AFTER, asserts the search request carries `Auth-Token: rutracker:cookie:…`; RED when `withAuth` reverts to build-time-only.

Fix commit: this commit.

HONEST SCOPE NOTE (§6.J / §6.AK — the keystone is NOT yet green): device re-runs proved A+B are necessary but NOT sufficient for the `--backend goapi` keystone, because in that harness the external Go API endpoint is added KEYLESS (no `Lava-Auth`), so `GET /v1/providers` 401s → the registry falls back to bundled providers → rutracker search resolves to the BUNDLED direct

`RuTrackerClient`, which never routes through the Go API (so the A+B path is not exercised there) and itself fails. A+B are the correct, verified fix for the `ApiBackedTrackerClient` session-token propagation (exercised by the on-device api-app backend + a keyed goapi); the keyless-endpoint→bundled-fallback + bundled-search-failure layers are a SEPARATE open investigation (evidence: `.lava-ci-evidence/autonomous-qa/2026-07-02/goapi/` + `docs/autonomous-qa/GOAPI-KEYSTONE-DEEP-DIAGNOSIS-2026-07-02.md`). This is documented as open, NOT claimed as fixed.

2026-07-02 — rutracker provider totally broken: client requested brotli but never decoded the response body

Discovered by: the autonomous-QA keystone (`run-matrix --backend goapi --subsets rutracker --queries 1080p`) — verdict SKIP; logcat showed `GetCurrentProfileUseCase.parseUserId: rutracker logged-in user-id not found` → onboarding login returned `ServiceUnavailable` and the flow could not proceed.

Root cause (CONFIRMED with physical evidence): both rutracker HttpClient paths — `TrackerClientModule.provideRuTrackerHttpClient` (on-device Hilt path) and `RuTrackerHttpClientFactory.create` (clone path) — set a **manual** `defaultRequest { header("Accept-Encoding", "gzip, deflate, br") }`. Ktor's OkHttp engine only transparently decompresses a Content-Encoding it negotiated **itself**; a manually-set Accept-Encoding makes OkHttp hand back the **raw** compressed bytes. rutracker replies `Content-Encoding: br` (brotli), so `RuTrackerInnerApiImpl.mainPage()` (and `search`/`browse`/`topic`) called `bodyAsText()` on undecoded brotli → garbage → every Jsoup selector matched nothing → `parseUserId` threw. `login()` survived only because it reads the session token from the `Set-Cookie` **header**, not the body, so the total breakage masqueraded as a transient `ServiceUnavailable`.

Physical evidence: curl replicating the app exactly (manual `Accept-Encoding: gzip, deflate, br`, no client decode) → `Content-Encoding: br`, raw body first bytes `5b8c9831…` (not HTML), `#logged-in-username` absent; brotli-decoded → valid logged-in page with `#logged-in-username` carrying `u=<id>`. Control: OkHttp's transparent `gzip` and identity both decode to the logged-in page. Eliminated (each tested): login charset (UTF-8 vs cp1251 — both authenticate), cookie/token handling (bb_session alone yields the logged-in page), stale selectors (work once decoded), missing `redirect` form field, wrong credentials. Full forensic: `.lava-ci-evidence/sixth-law-incidents/2026-07-02-rutracker-brotli-undecoded-body.json`.

Affected files / fix: removed the manual `Accept-Encoding` header from both `core/tracker/client/src/main/kotlin/lava/tracker/client/di/TrackerClientModule.kt` and `core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/RuTrackerHttpClientFactory.kt`, so OkHttp adds its own `Accept-Encoding: gzip` and decompresses transparently. Only rutracker set a manual header; rutor/kinozal/nnmclub were already transparent.

Verification test: `core/tracker/rutracker/src/test/kotlin/lava/tracker/rutracker/impl/RuTrackerBodyDecompressionRegressionTest.kt` — MockWebServer serves a gzip-encoded logged-in index; the real client + `RuTrackerInnerApiImpl.mainPage()` must return DECODED HTML containing the logged-in marker. Reproduce-first (§6.T.1): with the manual `Accept-Encoding` restored the test FAILS (raw bytes, marker absent); removed, it PASSES. Rehearsal run captured.

2026-07-02 — hermetic test bumped real CLAUDE.md mtime → spurious markdown-export-sync failures blocking every push

Root cause: `tests/check-constitution/test_covenant_114_propagation.sh` (the CM-COVENANT-114 falsifiability test) mutates the **real** repo-root `CLAUDE.md` (deletes the `11.4.134` anchor), runs the gate, then restores the original via a `trap` using a **plain `cp`**. Plain `cp` stamps the destination mtime to "now", so after this test ran, `CLAUDE.md` was newer than its generated `CLAUDE.html` / `CLAUDE.pdf` siblings. A LATER hermetic test in the same `scripts/ci.sh` run — `test_verify_all_rules.sh::test_clean_tree_passes` (runs after it; `v` > `c`) — re-invokes the full verify-all sweep, whose `markdown-export-sync` (§11.4.65) gate then reported the CLAUDE exports STALE and exited 1. Net effect: `ci.sh --changed-only` — and therefore **every `git push`** — failed at the hermetic phase, with a *different-looking* gate failing depending on run order (markdown-export in the direct sweep, or the hermetic phase via the re-run sweep). All one root cause: a non-hermetic test perturbing shared mtime state that feeds a later mtime-based gate.

Affected files: `tests/check-constitution/test_covenant_114_propagation.sh` (backup + `trap` restore, ~lines 80-90).

Fix: use `cp -p` for BOTH the backup and the `trap` restore so the test preserves `CLAUDE.md`'s ORIGINAL mtime and leaves shared repo state exactly as found. A hermetic test MUST NOT perturb repo state — mtime included.

Verification test/challenge: reproduce-first — regenerated CLAUDE exports (md < exports), ran the covenant test, confirmed `CLAUDE.md` mtime **UNCHANGED** before/after (it was bumped before the fix), then ran the previously-failing `test_verify_all_rules.sh` and full `scripts/ci.sh --changed-only` → **EXIT 0, "All --changed-only gates passed"** (hermetic phase now runs through `tests/vm-distro`). Peer tests `test_canonical_root_and_upstreams.sh` + `tests/pre-push/check8_test.sh` verified SAFE (they write CLAUDE.md only inside `mktemp -d` fixtures, never the real repo).

Fix commit: (this commit) **Forensic anchor:** surfaced while pushing the 2026-07-02 rutor fixture-refresh

- gate-fix commits; the pre-push hook rejected 4× with seemingly-different gate failures that all traced to this single mtime-perturbation bug.

`Classification`: project-specific (Lava hermetic-test hygiene; the underlying "hermetic tests must not perturb shared state" principle is universal).

2026-06-25 — display/onboarding video-sweep batch (issues #4 / #6 / #7 / #8 / #9)

Five display/onboarding defects from the operator's 2026-06-25 issue video. Each fix is root-caused to a `file:line`, reproduce-first tested (where it has logic), and falsifiability-rehearsed.

#4 [HIGH] — raw lowercased provider ids in search-RESULT filter chips

Root cause: The search-result chip renders `providerDisplayNames[pid] ?: pid` (`feature/search_result/.../SearchResultScreen.kt:377,383`) — display-name-correct WITH a raw-id fallback. The display name flows from `MultiSearchEvent.ProviderStart(id, descriptor.displayName)` (`core/tracker/client/.../LavaTrackerSdk.kt:826`). For an API-catalogue provider that descriptor is a `RemoteTrackerDescriptor` whose `displayName` is whatever the lava-api-go `/v1/providers` payload supplied — for these providers, the RAW id (or blank), so the chip showed "archiveorg"/"torrentdownloads"/"yts"/"kinozal". The bug

is NOT in the chip render (which is correct); it is the display name being a raw id at the catalogue SOURCE. **Affected files (SHARED/CORE source — per task, the chip render in `feature/search_result` was NOT edited and is flagged for the search subagent):** `core/data/src/main/kotlin/lava/data/provider/ProviderCatalogRepository.kt` (new `friendlyDisplayName(id, serverDisplayName)` + `toRemoteDescriptor` now humanizes), `core/network/api/.../dto/ProviderDescriptorDto.kt` (`displayName` defaulted to `""` so a missing field no longer drops the provider). **Fix:** the single catalogue→descriptor adapter humanizes a blank-or-equal-to-id `display_name` (split on _/-/./space → Title-Case), upgrading every downstream consumer (search chips, onboarding subtitles, provider-config rows) at once. A genuine server-supplied friendly name (≠ id, non-blank) is preserved. **Verification test:** `core/data/src/test/.../provider/ProviderDisplayNameTest.kt` (6 tests). **GREEN** `:core:data:testDebugUnitTest --tests lava.data.provider.ProviderDisplayNameTest` + `:core:network:api:test`. **Falsifiability:** echo-back mutation (`return serverDisplayName.ifBlank { id }`) → 5/6 FAIL ("Archiveorg" expected, "archiveorg" got). Reverted.

#6 [MEDIUM] — Welcome "N providers available" contradicted by the ~12-entry picker

Root cause: `OnboardingScreen.kt:136` passed `providerCount = state.providers .size` to WelcomeStep. `state.providers` is the BUNDLED list loaded at init (`OnboardingViewModel.loadProviders`, ~4 verified+apiSupported providers). In the ApiSelection flow, after the user picks an API, `fetchAndPopulateProviders` → `trackerRegistry.populateFrom(catalogue)` → `loadProviders()` repopulates the list to ~12. Welcome (which precedes API selection) showed the premature bundled count. **Affected files:** `OnboardingState.kt` (new `welcomeProviderCount: Int?`), `OnboardingViewModel.kt` (`loadProviders` sets it `null` when `apiSelectionEnabled`, else `items.size`), `steps/WelcomeStep.kt` (nullable param; `null` → count-free copy "Multiple content providers available"), `OnboardingScreen.kt:136`. **Fix:** count==list by construction — a concrete number is shown ONLY in the legacy Welcome→Providers flow where it IS the next screen's list; in the ApiSelection flow Welcome omits the (unknowable-yet) number. **Verification test:** `OnboardingViewModelVideoFixesTest` (2 tests). **GREEN. Falsifiability:** drop the `if (apiSelectionEnabled) null` guard → "ApiSelection flow MUST NOT pin a premature Welcome count; was 2 expected null". Reverted.

#7 [MEDIUM, UNCONFIRMED → resolved] — `lava.app:7777` preset + "On this network" label

§6.R verdict: NOT a violation. The `lava.app:7777` preset is NOT a hardcoded source literal. It originates in `.env.example:176` (`LAVA_DEFAULT_CLOUD_API=https://lava.app:7777`, a placeholder by definition), flows through `app/build.gradle.kts:101` `buildConfigField("DEFAULT_CLOUD_API", env["LAVA_DEFAULT_CLOUD_API"])`, is read by `CloudApiModule.defaultCloudApi() = BuildConfig.DEFAULT_CLOUD_API`, and parsed by `CloudApiDefaults.defaultsFrom()`. Every `7777`/`lava.app` literal in tracked `.kt` production source is in test files (§6.R-exempt) or comments. This is the §6.R-compliant config pattern, not a hardcoded literal. **Label bug (real, fixed):** the cloud preset is an `Endpoint.GoApi(platform=null)` rendered via `displaySubtitle()` → `"Lava API · ${discoveredApiLabel(null)}"` → `"Lava API · On this network"` — but a cloud/remote preset is NOT "on this network". **Affected files:** `steps/ApiSelectionStep.kt` (cloud preset rows now pass `subtitleOverride = CLOUD_SUBTITLE = "Cloud / remote server"`). **Verification test:** `OnboardingViewModelVideoFixesTest .cloud preset subtitle is honest and not on-this-network`. **GREEN.**

#8 [MEDIUM, UNCONFIRMED → real] — discovered API shown as raw `192.168.0.107:8443`

Friendly name IS available but was dropped. `DiscoveredEndpoint` carries the mDNS instance name (`.name`, e.g. lava-api-go's `LAVA_API_MDNS_INSTANCE`, default "Lava API"; the on-device app may publish its own) — `core/data/.../LocalNetworkDiscoveryServiceImpl.kt:108` reads `serviceInfo.serviceName`. But `OnboardingViewModel.startApiDiscovery` built `Endpoint.GoApi(host, port, platform, storage)` and DROPPED `hit.name`; the `Endpoint.GoApi` model has no name field. ApiSelectionStep then rendered `displayHostPort()` = raw `host:port`. Adding `name` to `Endpoint.GoApi` was rejected — it would change the model's `equals`/persistence identity that the menu-side dedup (`DiscoverLocalEndpointsUseCase: it == endpoint`) relies on. **Fix (display-only, identity-preserving):** thread the name through a side map `OnboardingState.discoveredApiNames: Map<"host:port", name>`; ApiSelectionStep renders the name as the PRIMARY label (host:port demoted to the subtitle) when present, else falls back to host:port unchanged. **Affected files:** `OnboardingState.kt`, `OnboardingViewModel.kt` (populate + symmetric resets + `discoveredApiNameKey`), `OnboardingScreen.kt`, `steps/ApiSelectionStep.kt` (`discoveredNames` param + `discoveredName` helper). Default-empty preserves all existing Challenge call sites (C26/C30) unchanged. **Verification test:** `OnboardingViewModelVideoFixesTest` (2 tests: VM map population +

discoveredName helper). **GREEN. Falsifiability:** drop the name-capture reduce → "discovered API name MUST be captured". Reverted.

#9 [LOW] — "Select all" silently enables captcha/ form-login providers

Root cause: OnboardingViewModel.onToggleAllProviders set every item selected = true on select-all, including providers whose authType != NONE && !supportsAnonymous (FORM_LOGIN/ CAPTCHA_LOGIN without an anonymous path) — bulk-selecting them strands the user on a Configure page they cannot pass without credentials they never entered. **Affected files:** OnboardingViewModel.kt (onToggleAllProviders + requiresNoCredentials() predicate). **Fix:** select-all enables ONLY no-credential-reachable providers (authType == NONE || supportsAnonymous); credential-required providers stay unselected so the user opts in explicitly (the informed tap that routes them through Configure). Deselect-all still clears the WHOLE list. **Verification test:** OnboardingViewModelVideoFixesTest (2 tests). **GREEN. Falsifiability:** select-all target = true for all → "credential- required provider MUST NOT be silently enabled by select-all". Reverted.

Fix commit: see git log on the worktree branch fix/display-onboarding-bugs-batch. Full onboarding unit suite 64/64 green; core:data provider suite + core:network:api test green. No version bump, no push (per task scope).

2026-06-25 — provider "Sync this provider" toggle crashes (SerializationException, prod 1.3.11(1075) RELEASE)

Root cause: feature/provider_config/build.gradle.kts applied only lava.android.feature + lava.android.library.compose — it did NOT apply the lava.kotlin.serialization convention plugin (which applies org.jetbrains.kotlin.plugin.serialization, the compiler plugin that GENERATES $serializer companions for @Serializable classes). The kotlinx-serialization-json RUNTIME + the encodeToString API leaked transitively into the module via the :core:domain dependency (core:domain applies the plugin), so ProviderConfigViewModel's json.encodeToString(WireToggle(...)) COMPILED — but no $serializer was ever generated for the module's own @Serializable wire classes, so at runtime kotlinx-serialization fell back to reflective serializer lookup and threw kotlinx.serialization.SerializationException: Serializer for class

'WireToggle' is not found. Settings → any provider → "Sync this provider" toggle → ProviderConfigViewModel.kt:92 crash. Two compounding factors: (1) the same defect was LATENT in all three wire classes — WireToggle (ToggleSync), WireBinding (BindCredential / UnbindCredential), WireMirror (AddMirror / RemoveMirror); (2) app/proguard-rules.pro had NO kotlinx-serialization keep rules, so the RELEASE R8 build would strip the generated $serializer even after the plugin was applied → release-only crash class.

Affected files:

- feature/provider_config/build.gradle.kts — added id("lava.kotlin.serialization").
- app/proguard-rules.pro — added kotlinx-serialization keep rules (generic @Serializable + $serializer rules + a targeted lava.provider.config.** keep for the prod-crash surface).
- feature/provider_config/src/test/kotlin/lava/provider/config/ProviderConfigWireSerializationTest.kt — new reproduce-first regression test (covers all 3 wire classes).

Fix: apply the project's serialization convention plugin (lava.kotlin.serialization, buildSrc id at buildSrc/build.gradle.kts:62) so the compiler plugin generates the $serializer companions for WireToggle / WireBinding / WireMirror; add R8 keep rules so the RELEASE build does not strip them. No production Kotlin source changed — the @Serializable annotations and encodeToString calls were already correct; only the missing build wiring + keep rules.

Verification test/challenge: ProviderConfigWireSerializationTest — drives the REAL ProviderConfigViewModel ToggleSync / BindCredential / AddMirror actions with a recording SyncOutbox, asserting the production json.encodeToString(Wire*(...)) path reaches enqueue with a VALID JSON payload (the user-observable "queued for sync" outcome) rather than crashing. Reproduce-first per §11.4.146.

RED (current/broken code, serialization plugin absent) — 3/3 FAIL:
kotlinx.serialization.SerializationException: Serializer for class 'WireToggle' is not found.
kotlinx.serialization.SerializationException: Serializer for class 'WireBinding' is not found.
kotlinx.serialization.SerializationException: Serializer for class 'WireMirror' is not found. → assertion: "ToggleSync MUST enqueue a SYNC_TOGGLE payload — none was recorded (serialization threw before enqueue). recorded=[]" (+ BINDING + USER_MIRROR). GREEN (after fix) — :feature:provider_config:compileDebugKotlin BUILD SUCCESSFUL,

`:feature:provider_config:testDebugUnitTest` BUILD SUCCESSFUL, 3 tests / 0 failures. The pre-existing `ProviderConfigViewModelTest` (3/0/0) also still passes — no regression.

RELEASE-variant R8-strip verification is OWED at the §6.Z device gate — a JVM unit test cannot exercise R8 resource/code shrinking. The `proguard-rules.pro` keep rules are the release-side fix; proving they hold requires the release APK on the gate.

Fix commit: (this commit, worktree branch — not pushed; main stream bumps to 1076) **Forensic anchor:** Crashlytics issue `eaa80c1486d2d5d7526346ece016e15a` (prod 1.3.11(1075) RELEASE); §6.O closure log `.lava-ci-evidence/crashlytics-resolved/2026-06-25-provider-sync-toggle-serialization.md`.

2026-06-16 — 3 circuit-breaker providers: transient timeout tripped the breaker (kinozal, nnmclub, rutracker)

Root cause: kinozal, nnmclub, and rutracker wrap their HTTP calls in a circuit breaker (`c.breaker.Execute(closure)`). The closure had no retry, so a single transient timeout/5xx returned an error to the breaker → counted as a consecutive failure. ~5 transients would OPEN the breaker (~10 s lockout) — strictly WORSE than the plain single-attempt gap, because one slow upstream response could lock out the user for 10 s.

Affected files: `internal/kinozal/client.go` (Fetch/FetchWithHeaders/PostForm), `internal/nnmclub/client.go` (4 methods), `internal/rutracker/client.go` (5 methods)

- each provider's `*_test.go` (nnmclub's `client_test.go` is new). Added `maxAttempts=3`/`retryBackoff=500ms` + a `doWithRetry` helper used INSIDE each breaker Execute closure.

Fix: retry transient failures (network/timeout or 5xx) INSIDE the breaker closure, so a recovered transient returns `nil` to the breaker (failure counter untouched, breaker stays Closed). Genuine post-retry failures still return one error to the breaker — real-outage counting is unchanged. Terminal errors (404/403/decode) never retried.

Verification test/challenge: per provider, `TestRetriesOnTransient5xx` (503-once-then-200 → recovers + asserts `breaker.GetFailures()==0` + `GetState()==StateClosed` — the load-bearing proof the transient did NOT trip the breaker) + `TestTerminalErrorNotRetried` (404 → 1 attempt, surfaced immediately). Falsifiability rehearsed all 3: bypassing the retry →

`TestRetriesOnTransient5xx` FAILS ("expected transient 503 to be retried into success, got err: upstream 503"); reverted → `ok`. gofmt + go vet clean.

Fix commit: (this commit) — completes provider retry-resilience for all 10 HTTP providers (this 3 + the 7 below). Same undistributed api-go cycle (2.3.31). **Forensic anchor:** nezha audit (3-subagent fan-out) of every provider HTTP client.

2026-06-16 — 6 sibling providers shared the tokyotosho single-attempt-no-retry gap (knaben, nyaa, bitsearch, torrentdownloads, gutenberg, flaresolverr)

Root cause: an audit of every provider HTTP client (triggered by the tokyotosho fix below) found the SAME single-attempt-no-retry class in 6 more providers: each issued one `http.Do` with no retry, so a transient upstream timeout/5xx surfaced a user-facing error even though the next request would succeed. Cited from source: knaben/client.go (single Do), nyaa/client.go, bitsearch/client.go, torrentdownloads/client.go, internal/gutenberg/client.go (getJSON + downloadBytes), internal/provider/flaresolverr/client.go (Get).

Affected files: the `client.go` of each of the 6 providers (refactored the inline fetch into a bounded-retry loop + a single-attempt helper classifying transient = network/timeout-from-http.Do OR 5xx; terminal = 404/403/401/decode never retried) + each provider's `*_test.go` (added retry + terminal-not-retried tests). flaresolverr uses maxAttempts=2 (its 90s solve budget is already long); gutenberg retries BOTH getJSON + downloadBytes via a shared helper.

Fix: same bounded-retry pattern as tokyotosho (maxAttempts=3 / 500ms backoff, ctx-bounded), adapted to each provider's response shape (RSS/JSON/HTML/latin-1).

Verification test/challenge: per provider, `TestSearch_RetriesOnTransient5xx` (+ download variant for gutenberg) and `TestSearch_TerminalErrorNotRetried`. Falsifiability rehearsed for ALL 6: bypassing the retry loop makes the retry test FAIL with the provider's exact `HTTP 503: ... unknown error` (e.g. `knaben: HTTP 503: provider: unknown error`); reverted → `ok`. go vet + gofmt clean.

Fix commit: (this commit) **Forensic anchor:** nezha real-tracker E2E surfaced tokyotosho; the systemic audit (3-subagent fan-out) surfaced the 6 siblings. Same api-go cycle (2.3.31).

2026-06-16 — tokyotosho curated provider: user-facing "unknown error" on slow upstream (no retry on transient failure)

Root cause: `internal/provider/curated/tokyotosho` issued a single HTTP request bounded by `DefaultTimeout = 20s`. Tokyo Toshokan's live latency is variable (measured on the nezha heavy-test node 2026-06-16: typical 4–8 s, occasionally >20 s). When a single request exceeded 20 s the client returned `provider: ErrUnknown` ("unknown error") to the user — even though the very next request would have succeeded. Surfaced by the `realtrackers` live test `TestLive_SearchReturnsRealMagnets`, which PASSED at 16:14 (16.5 s) and FAILED at 16:15 (the >20 s case), confirming external-latency intermittency, not a parser bug (direct curl probes all returned HTTP 200 with valid RSS).

Affected files: `lava-api-go/internal/provider/curated/tokyotosho/client.go` (added `maxAttempts`/`retryBackoff`; refactored the inline fetch into `fetchFeed` (bounded-retry loop) + `fetchFeedOnce` (single attempt classifying transient vs terminal)). Terminal errors (404/403/401/decode) are NOT retried.

Fix: retry transient failures (network/timeout or 5xx) up to 3 attempts with a 500 ms backoff, bounded by the caller's ctx. Converts transient upstream slowness into a successful search while leaving the user-visible result shape and the production `DefaultTimeout` unchanged.

Verification test/challenge: `tokyotosho_test.go::TestSearch_RetriesOnTransient5xx` (reproduce-first §6.T.1: 503-once-then-fixture → recovers, 2 results) + `TestSearch_TerminalErrorNotRetried` (404 → ErrNotFound after exactly 1 attempt). Falsifiability rehearsed: bypassing the retry loop → `TestSearch_RetriesOnTransient5xx` FAILS with "should recover via retry: tokyotosho: HTTP 503: provider: unknown error"; reverted → PASS.

Fix commit: (this commit) **Forensic anchor:** nezha.local heavy-testing enablement, real-tracker E2E run; operator chose "add bounded retry". Evidence: `.lava-ci-evidence/nezha/2026-06-16-system-boot.md`.

2026-06-14 — search "Something went wrong" across providers via the on-device API (ApiBackedTrackerClient used the strict client + no per-endpoint key)

Root cause: the dynamic `ApiBackedTrackerClient` (the client that issues `GET|POST /v1/{provider}/{op}` against the chosen lava-api-go / on-device api-app) was wired in `core/tracker/client/.../di/TrackerClientModule.kt:275/302` with the **unqualified `okHttpClient`** — which (per `NetworkModule.kt:101`) uses the system trust store only and is explicitly documented "MUST NOT be used for LAN endpoints." The on-device api-app serves over HTTPS with a **self-signed LAN cert**, so every search failed at the TLS handshake (`CertPathValidatorException`); and even past TLS, `ApiBackedTrackerClient` attached **no per-endpoint Lava-Auth key**, so the auth-gated `/v1/{provider}/search` returned **HTTP 401**. Either way `getString` threw → `SearchResultViewModel` mapped the Throwable to `error_something_goes_wrong` ("Something went wrong, please try again"). This is the exact SP-3.1 / Defect-A class that was fixed for the `/providers` catalogue fetch (`ProviderCatalogRepository` got the `@Named("lan")` client + `goApi.key`) but **never fixed for the per-provider search/browse/topic/download/login** path.

Affected files: `core/tracker/client/.../di/TrackerClientModule.kt` (factory → `@Named("lan")` client + `@Named("authFieldName")` + `authKey = ApiBaseUrlHolder.currentKey()`); `core/tracker/client/.../ApiBackedTrackerClient.kt` (+`authFieldName`/`authKey` ctor params + `Request.Builder.withAuth()` on every request); `core/tracker/client/.../ApiBaseUrlHolder.kt` (now holds the per-endpoint key); `core/domain/.../RepopulateProvidersOnStartupUseCase.kt`

- `ActiveApiBaseUrlActivator` (cold-start threads the key); `feature/onboarding/.../OnboardingViewModel.kt` (`ApiBaseUrlHolder.set(apiBaseUrl, goApi.key)`); `app/.../StartupProvidersModule.kt`.

Fix: the dynamic client now uses the permissive-TLS `@Named("lan")` client (accepts the self-signed LAN cert) and attaches the active endpoint's per-instance key as the Lava-Auth header on EVERY `/v1` request — mirroring the proven Defect-A pattern. The key is threaded from onboarding AND the cold-start re-populate.

Verification test/challenge: `ApiBackedTrackerClientTest.search_attachesPerEndpointAuthKey_soAuthGatedApiReturnsResults` (real client + MockWebServer that 401s without the key → asserts a real result row

- the `Lava-Auth: k` header on the wire) + … `search_withoutAuthKey_throwsOnAuthGatedApi` (discriminator) + `RepopulateProvidersOnStartupUseCaseTest` (asserts the cold-start activator receives the key). Falsifiability (re-performed): neuter `ApiBackedTrackerClient.withAuth()` → the search test FAILS with HTTP 401; reverted → green. Device-level proof (search on the VM against the real api-app) is the final gate via the §6.AE/ HelixQA video pass.

Fix commit: (this commit).

Forensic anchor: operator report (2026-06-14) — "onboarded RuTracker, YTS, Kinozal; searched 'prince' via the Android API with all 3 selected → 'Error, Something went wrong, please try again'."

Update (2026-06-14) — L5 root cause FOUND + fixed (api-app `ApiKeyProvider`), release scope now client + api-app, status: on-device verification in progress — NOT yet shipped. After the client-side fixes above (now framed as L1–L4 of a 5-layer cascade — see `docs/issues/2026-06-14-search-multilayer-rootcause.md`), search STILL returned 401 on a clean matched pair (`granted=true`, engine up). The load-bearing fifth layer (L5) was an **api-app** defect, not a client one: `api-app/src/main/kotlin/lava/api/app/handoff/ApiKeyProvider.kt` cached its key and port resolver lambdas in `ContentProvider.onCreate()`, gated on the `ApiApplication.controllerHolder/keyStoreHolder` being non-null, on the FALSE inline comment "ContentProvider.onCreate runs AFTER Application.onCreate." Android runs `ContentProvider.onCreate` **BEFORE** `Application.onCreate`, and the holders are set inside `ApiApplication.onCreate` (lines 46-49) — so at cache time the holders were **always null**, the resolver lambdas stayed `{ null }` for the whole process, and the provider served an **empty cursor**. The full key-loss chain: client `ApiKeyClient.read()` → null → `withLocalApiKeyIfMissing` → keyless endpoint → `ApiBaseUrlHolder.set(url, null)` → `ApiBackedTrackerClient` built with `authKey=null` → no `Lava-Auth` header → every `/v1/{provider}/search` 401. Public routes (`/providers`, `/health`) need no key, so they worked — which is why search NEVER worked while everything else did. The existing `withFakes` unit test was a **bluff**: it injected the resolver lambdas through a test-only seam, bypassing the real `onCreate` → holder path where the bug lives, so it passed green against the empty-cursor production path. **Fix (applied):** `ApiKeyProvider` resolves the holders + the engine Running state **LAZILY per `query()`** (`resolveRunningKey()` / `resolveRunningPort()` defaults), removing the `onCreate`-ordering dependency. **On-device pinpoint:** `.lava-ci-evidence/search-verification/2026-06-14-

`keyloss-pinpoint.md` (LAVAKEYDBG logcat `withLocalApiKey readerSet=true readLen=-1` = key read returned null at the source). **Verification OWED:** a real-holder regression test driving the actual `onCreate` ordering (no `withFakes` seam) so the empty-cursor defect fails RED before the fix. **Release scope change:** 1.3.9 is NO LONGER client-only — the L5 fix is in the api-app, so **client 1.3.9-1066 + api-app 0.2.9-14 ship together** (§6.Y bump applied). **Status: fix applied, on-device verification in progress — NOT yet shipped.** Search is NOT claimed to work on-device until the device gate (authed `/v1/{provider}/search` → 200 + result rows) is GREEN.

2026-06-13 — curated TPB + Torrents-CSV providers single-domain rotation risk (no mirror failover)

Root cause: the embedded curated providers `thepiratebay` (apibay.org) and `torrentscsv` (torrents-csv.com) each hardcoded a SINGLE base domain in their `Client`. Public-tracker domains rotate frequently under takedown pressure — the companion YTS provider was caught by exactly this on 2026-06-13 (yts.mx went NXDOMAIN on public DNS while yts.bz still served HTTP 200). A single-domain client goes silently unreachable the moment that one domain drops, so the provider would appear in the on-device `GET /providers` catalogue but every search would error for the user — a reachability bluff waiting to happen.

Fix: refactored both clients to the YTS mirror-failover pattern — `NewClientWithMirrors(DefaultBaseURLs)`, a first-live-mirror-wins loop in `Search`, and a `perAttemptTimeout` (8s) cap per mirror so one dead/hanging host cannot stall the whole search. `New()` registration now goes through the mirror list. The failover ARCHITECTURE is the load-bearing change; the production `DefaultBaseURLs` for each provider deliberately contains only ONE host because live HTTP probing on 2026-06-13 found exactly one real endpoint per provider:

- **TPB:** apibay.org/q.php = HTTP 200 JSON; the TPB frontend proxies (thepiratebay.org/apibay 302→SPA, tpb.party / thepiratebay10.xyz 404 /q.php) expose NO compatible JSON API. apibay.org is TPB's canonical internal API.
- **Torrents-CSV:** torrents-csv.com/service/search = HTTP 200 JSON; torrents-csv.ml = 404, torrents-csv.org / api.torrents-csv.com = NXDOMAIN. torrents-csv.com is the canonical public instance (git.torrents-csv.com is the Gitea code repo, not a search API). Per §6.L, fabricating dead `.xyz`/`.party`/`.ml`/`.org` mirror entries would be a bluff (they'd only add latency + failover noise), so each list stays a real single-element slice; a future real mirror is a one-line addition with no client change.

YTS itself genuinely has 4 live mirrors and keeps its multi-host list.

Affected files: `lava-api-go/internal/provider/curated/thepiratebay/client.go`, `.../thepiratebay/provider.go`, `.../thepiratebay/thepiratebay_test.go`, `.../torrentscsv/client.go`, `.../torrentscsv/provider.go`, `.../torrentscsv/torrentscsv_test.go`, `core/apiengine/src/main/resources/api-source.hash` (recomputed for the embed).

Verification test/challenge: new fast fixture (httptest, no live net) tests in each package: `TestSearch_FailsOverToHealthyMirror` (dead 503 first mirror → returns the SECOND healthy mirror's real parsed `SearchItem` rows — primary assertion on user-visible info_hash, not a call count) + `TestSearch_AllMirrorsDownSurfacesError` (all mirrors error → no fake empty success). `go test ./internal/provider/curated/...` = all `ok`.

Bluff-Audit: TPB `TestSearch_FailsOverToHealthyMirror` — Mutation: add `return nil, lastErr` after the first loop iteration in `Search` (break failover after first mirror). Observed: `Search across [dead, healthy] mirrors should fail over, got error: thepiratebay: HTTP 503: provider: unknown error`. Torrents-CSV `TestSearch_FailsOverToHealthyMirror` — same mutation. Observed: `Search across [dead, healthy] mirrors should fail over, got error: torrentscsv: HTTP 503: provider: unknown error`. Reverted: yes (both re-run `ok`).

Fix commit: (this worktree commit — see git log)

Forensic anchor: YTS domain-failover fix `.lava-ci-evidence/bluff-hunt/2026-06-13-yts-domain-failover.json`; the same single-domain rotation class applied to the two sibling curated providers.

2026-06-13 — Tokyo Toshokan curated provider added (Defect B — anime/Asian-media RSS, honest CapSearch)

Type: feature (curated embedded provider), logged here per §6.T.4 because it extends the user-reachable provider catalogue.

What: new compiled-in curated provider `tokyotosho` (`lava-api-go/internal/provider/curated/tokyotosho/`) sourcing the public Tokyo Toshokan RSS search feed (`https://www.tokyotosho.info/rss.php?terms=<q>`), AuthType NONE / SupportsAnonymous true, capabilities SEARCH + MAGNET_LINK. Registered in `curated.RegisterAll` (both startup paths). Broadens the anime/Asian-media niche alongside `nyaa`.

Wire-shape specifics (differ from nyaa): the magnet is embedded in each item's `<description>` CDATA HTML (`<a href="magnet:?xt=urn:btih:<HASH>…">`), extracted by regex; the info_hash is BASE32 (32 chars), not 40-hex; the RSS carries NO seeders field, so `Seeders` defaults to 0 (documented, honest absence); size is decimal SI (`113.89MB` → 113890000 bytes), not nyaa's IEC.

§6.E honesty (the load-bearing check): the `terms=` parameter genuinely filters — verified LIVE 2026-06-13 (real HTTP): `terms=naruto` → 93% of titles contain "naruto"; `terms=bleach` → 98% contain "bleach"; the empty feed contains neither; naruto vs bleach result sets are near-disjoint (2 of ~140 overlap). So CapSearch is honest — NOT the EZTV/Knaben-`search_type` no-op-query bluff. The `//go:build realtrackers TestLive_QueryActuallyFilters` encodes both proofs (term-match-rate ≥ 50% + ≤ 50% cross-query info_hash overlap) and PASSED against the real endpoint.

Affected files: `lava-api-go/internal/provider/curated/tokyotosho/{client.go,provider.go,tokyotosho_test.go,live_realtrackers_test.go,testdata/tokyotosho_naruto.xml}` (new); `lava-api-go/internal/provider/curated/curated.go` (one import + one `r.Register` line); `core/apiengine/src/main/resources/api-source.hash` (recomputed).

Verification: fixture test `tokyotosho_test.go` (real Client + httptest.Server serving captured real RSS bytes; asserts parsed title/infohash/magnet/size/no-magnet-filter)

- live `live_realtrackers_test.go` (`TestLive_SearchReturnsRealMagnets` + `TestLive_QueryActuallyFilters`, both PASS against the real endpoint). `go test ./internal/provider/curated/...` = all `ok`.

Bluff-Audit (§6.J clause 2 / Seventh Law clause 1):

- Mutation: `extractInfoHash` returns `""` unconditionally (magnet extraction off) → `TestSearch_ParsesFixture` FAILED `got 0 results, want 2 (the no-magnet row must be filtered)`. Reverted.
- Mutation: send the query as `xterms` not `terms` (query-not-sent under the honored name) → `TestSearch_SendsTermsParam` FAILED `Search: tokyotosho: HTTP 400: provider: unknown error`. Reverted.
- Mutation: `MB` multiplier `1e6 → 1<<20` (IEC instead of decimal SI) → `TestSearch_ParsesFixture` FAILED `sizeBytes = 119422320, want 113890000 (113.89 MB decimal)`. Reverted.

Fix commit: see this commit (worktree).

2026-06-13 — onboarding catalogue fetch 401 against a freshly-discovered API (`GET /providers` auth-gated)

Root cause: `lava-api-go/internal/router/router.go` registered `engine.GET("/providers", …)` AFTER `engine.Use(auth.GinMiddleware())` (and the `auth.NewMiddleware` chain). The provider catalogue is the endpoint the Android client fetches during ONBOARDING against an API it has just discovered on the LAN — at which point the client holds no pre-shared `Lava-Auth` key with that API. The auth gate therefore rejected the fetch with HTTP 401 → `OnboardingViewModel` `fetchAndPopulateProviders` caught the failure and surfaced `PROVIDER_CATALOG_FALLBACK_NOTICE` ("Couldn't reach the selected API — showing bundled providers"), so a perfectly reachable API showed the bundled fallback. This is distinct from Defect A (2026-06-12, a TLS+missing-key client problem on the *client* side); Defect A's fix made the client send the key, but the api-app's allowlist need not include a just-discovered client at all — the catalogue is public metadata and must not be gated in the first place.

Affected files: `lava-api-go/internal/router/router.go` (registration moved before the auth middleware, with rationale comment + a NOTE where it used to live); `lava-api-go/internal/router/router_config_wiring_test.go` (+`TestBuild_ProvidersOpenWithFullAuthChain`); `lava-api-go/tests/contract/providers_public_auth_boundary_test.go` (new, real-binary 3-way boundary); `feature/onboarding/src/test/kotlin/lava/onboarding/OnboardingViewModelDynamicProvidersTest.kt` (+replace-not-merge success test + no-banner-on-success assertion); `core/apiengine/src/main/resources/api-source.hash` (recomputed).

Fix: register `GET /providers` BEFORE the auth middleware, alongside `/health`+`/ready`. It is public, non-sensitive provider metadata (ids, capabilities, authType, baseUrls — no credentials, no user data). Per-provider operations under `/v1/:provider/…` remain fully auth-gated. The standalone binary and the embedded api-app share the exact `router.Build`, so the one edit fixes both surfaces (DRY; a divergent embed router would be a §6.J bluff vector).

Verification test/challenge: server `TestBuild_ProvidersOpenWithFullAuthChain` (unit) + `TestProviders_PublicCatalogue_PerProviderStillGated` (real-binary e2e, proves unauth `/providers`→200 while unauth `/v1/…`→401 and authed `/v1/…` crosses the gate); client `OnboardingViewModelDynamicProvidersTest` (success → exactly the API's set, bundled-absent, no banner; failure → banner + non-blank). Every layer's falsifiability mutation re-performed by the

main stream (not trusted from the subagent worktree): gate `/providers` → e2e sub-test A 401; drop `populateFrom` → replace test FAILED; success-emits-banner → assertNull FAILED. All reverted GREEN.

Fix commit: `9ae9ab90` (server fix) + `132d1b07` (real-binary e2e) + `99e5893a/6ce100bc` (client full-flow tests).

Forensic anchor: operator real-device report (2026-06-13) — "choose discovered API 192.168.0.107:8443 → next screen shows 'Couldn't reach the selected API — showing bundled providers'". Crashlytics non-fatal `47b000d5` (`provider_catalog_fetch_failed`, HTTP 401).

2026-06-12 — provider-catalogue fetch fails self-signed-LAN TLS handshake (Defect A — "only 4 providers")

Root cause: `ProviderCatalogRepository` injected the *unqualified* strict-TLS `OkHttpClient` (system trust store only) and sent no auth key. The on-device api-app serves `GET /providers` over a self-signed LAN cert behind the `Lava-Auth` gate, so every real-device fetch died at the TLS handshake (`CertPathValidatorException: Trust anchor … not found`) → `Result.failure` → onboarding fell back to the 4 bundled `verified && apiSupported` descriptors. **Real-device proof:** Crashlytics NON_FATAL `042b9b61` (`provider_catalog_fetch_failed`), 1.3.3-1060, Galaxy S23 Ultra / Android 16. **Affected files:** `core/data/.../ProviderCatalogRepository.kt` (inject `@Named("lan")` client + `@Named("authFieldName")`; `fetchProviders(apiBaseUrl, authKey)` + `withAuthKey`), `core/domain/.../FetchProvidersUseCase.kt` (thread `authKey`), `feature/onboarding/.../OnboardingViewModel.kt` (pass `Endpoint.GoApi.key`), `core/data/build.gradle.kts` + `gradle/libs.versions.toml` (okhttp-tls for self-signed-HTTPS tests). **Fix:** route the catalogue fetch through the permissive LAN client (accepts the self-signed cert) + attach the endpoint's per-instance key — mirrors the existing `NetworkApiRepositoryImpl.getApi()` `Endpoint.GoApi` path. **Verification test/challenge:** `ProviderCatalogRepositoryTest` rewritten to cross a real self-signed-HTTPS + auth-gated MockWebServer (6/6; `strictClientFailsTheHandshake` = codified pre-fix mutation; `missingAuthKeyIsRejectedByTheGate` proves the header is load-bearing); `OnboardingViewModelDynamicProvidersTest` green. **Fix commit:** `0deb54e7` **Forensic anchor:** operator "only 4 providers in onboarding wizard"; §6.O closure log `.lava-ci-evidence/crashlytics-resolved/2026-06-12-provider-catalog-fetch-tls.md`. **Scope note:** Defect B (embedded Jackett in the api-app) still

OPEN — the client now shows whatever the api-app serves, but the api-app serves only native providers until embedded Jackett lands.

2026-06-02 — api-app-ondevice-challenges-three-defects + harness-isolation

The Containers `emulator-matrix` CLI gained a generic `--gradle-module` flag (Containers commit `9a61a153`); the Lava glue `scripts/run-api-app-challenge-matrix.sh` now forwards it so the `:api-app` Compose UI Challenges (C01–C04) finally **execute** against the `:api-app` module instead of a 0-test false-green against `:app`. Running them for real surfaced **three latent product defects** (all previously invisible because the tests had never actually run) plus a test-harness isolation flaw. All proven fixed: clean sequential 4/4 green on a cold-booted Pixel_8/API35 via the Containers runner (host-direct+HVF, `gating=true`).

Root cause 1 (C02 — mDNS service-name shadow → API35 crash): in `NsdMdnsAdvertiser.register()`, `NsdServiceInfo().apply { this.serviceName = serviceName }` — inside `apply{}` the implicit receiver is the `NsdServiceInfo`, whose own `serviceName` property **shadowed** the advertiser's constructor val, so the RHS `serviceName` resolved to the receiver's (null) value. The service registered with an EMPTY name → Android 15 `NsdManager.validateService` threw `IllegalArgumentException` ("The service name or the service type is missing") → uncaught → engine start crashed → instrumentation process crash (C01 ok, C02 crashed, C03/C04 never ran).

Root cause 2 (C03 — cross-test native-engine pollution): the Go embed's `internal/mobile.current` is process-global, but Hilt rebuilds the `@Singleton ApiEngineController` per test. C02 starts the engine and never stops it; C03 then tries to Start → `mobile.Start` sees `current != nil` → "already running" → C03's start→Running wait times out.

Root cause 3 (C04 — notification restart-after-stop): two bugs. (a) `ApiEngineController.restart()` called `stop()` first unconditionally; after a Stop the engine is already stopped → `mobile.Stop` returns "no server running" → `Error` state → `restart()` bailed before `start()`. (b) `ApiEngineService`'s state collector called `stopSelf()` on ANY `Stopped`, including the INITIAL `Stopped` a freshly-recreated Service sees while handling an ACTION_RESTART — destroying the Service and cancelling the restart coroutine before it could drive the engine up.

Root cause 4 (harness isolation flakiness): the foreground `ApiEngineService` is a process-singleton that outlives the per-test Hilt `@Singleton` controller, so a stale Service from a prior test intercepted the next test's start/notification actions (driving the wrong, stale controller) — producing run-to-run inconsistent failures. Plus the on-device `OnDeviceApiClient` 15s readTimeout flaked the HTTP/2 stream (`SocketTimeoutException`) on the emulator's first cold-serve.

Affected files:

- `submodules/containers/.../cmd/emulator-matrix` + `pkg/emulator/*` (the `--gradle-module` flag, Containers commit `9a61a153`)
- `scripts/run-api-app-challenge-matrix.sh` (forward `--gradle-module`)
- `api-app/src/main/kotlin/lava/api/app/service/NsdMdnsAdvertiser.kt` (RC1)
- `api-app/src/main/kotlin/lava/api/app/control/ApiEngineController.kt` (RC3a)
- `api-app/src/main/kotlin/lava/api/app/service/ApiEngineService.kt` (RC3b)
- `api-app/src/androidTest/.../Challenge0{1,2,3,4}*Test.kt` (RC2 + RC4 `@Before` reset + Service teardown)
- `api-app/src/androidTest/.../OnDeviceApiClient.kt` (RC4 timeout)
- `lava-api-go/internal/mobile/restart_repro_test.go` (NEW Go same-port-restart coverage — proved the Go embed restart is innocent)

Fix:

1. RC1 — capture `serviceName`/`serviceType` into LOCALS before the `apply{}` (kills the shadow) + wrap `registerService` in `runCatching` so a future platform rejection degrades gracefully instead of crashing the engine.
2. RC2 + RC4 — each Challenge `@Before` resets the process-global native engine (`NativeApiEngine().stop()`) AND kills any stale foreground Service (`stopService`).
3. RC3a — `restart()` skips `stop()` when already `Stopped` (just `start()`).
4. RC3b — the Service collector self-stops only on a genuine running→stopped transition (`sawRunning` flag), never on the initial `Stopped`.
5. RC4 — `OnDeviceApiClient` connect/read timeouts 10/15s → 20/30s.

Verification test/challenge: `:api-app` Challenge C01–C04 EXECUTED green on cold-booted Pixel_8/API35 via the Containers runner (gating=true, host-direct+HVF). Evidence: `.lava-ci-evidence/phase-e-api-app/2026-06-02T11-32-48Z-gate/` (+ a confirmation run). Each fix falsifiability-rehearsed; the gate itself is the load-bearing on-device acceptance gate (§6.AE / §6.Z). **Fix commit:** `0deb54e7` (parent) + Containers `9a61a153`. **Forensic**

anchor: the `--gradle-module` flag made the never-before-run `:api-app` Challenges execute for real — the textbook §6.J/§6.L payoff: three real product defects had been invisible behind a 0-test false-green.

2026-05-06 — phase1-distribute-three-bugs

The first real-distribute of Lava-Android-1.2.7-1027 + lava-api-go-2.1.0 exposed three bugs that landed in commit `e947081`:

Root cause 1: `/health` and `/ready` endpoints were registered AFTER the auth middleware in `lava-api-go/cmd/lava-api-go/main.go buildRouter`, so the orchestrator's liveness probe got 401 → restart loop.

Root cause 2: `core/network/impl/.../LavaAuthBlobProvider.kt` was declared `internal`, but the Phase 11 build-time-generated `lava.auth.LavaAuthGenerated` class lives in `:app`'s source set — different module → cannot access internal interface → compile failure.

Root cause 3: `scripts/distribute-api-remote.sh` + `deployment/thinker/thinker-up.sh` did NOT ship the operator's local `.env` `LAVA_AUTH_*` + `LAVA_API_HTTP3_*`/`BROTLI_*`/`PROTOCOL_*` values to the thinker.local container. The new 2.1.0 binary required `LAVA_AUTH_FIELD_NAME` + `LAVA_AUTH_HMAC_SECRET` at boot → crash-loop.

Affected files:

- `lava-api-go/cmd/lava-api-go/main.go`
- `core/network/impl/src/main/kotlin/lava/network/impl/LavaAuthBlobProvider.kt`
- `scripts/distribute-api-remote.sh`
- `deployment/thinker/thinker-up.sh`

Fix:

1. Register `/health` + `/ready` BEFORE the auth chain.
2. Drop `internal` from `LavaAuthBlobProvider`.
3. Distribute script merges operator's `.env` auth/transport block into a temp env file before scp; thinker-up iterates the variables and passes each as `-e VAR=$VAR` to `podman run`.

Verification test/challenge: post-fix smoke test in `scripts/distribute-api-remote.sh`'s 60-second `/health` wait (passes); `curl -fsSk https://thinker.local:8443/{health,ready}` return

`{"status":"alive"}` / `{"status":"ready"}` (passes); auth gate returns 401 with `{"error":"unauthorized"}` for missing header (passes — fail-closed posture confirmed).

Fix commit: `e947081`

Forensic anchor: the very first `bash scripts/distribute-api-remote.sh` after the api-go-2.1.0 build showed `lava-api-go: config: config: LAVA_AUTH_FIELD_NAME is required` in a tight loop, with the orchestrator marking the container `(unhealthy)` and triggering the restart loop. Operator ran the distribute, got the lock-up output, immediately surfaced the issue.

2026-05-12 — c03-anonymous-toggle-checkauth-bluff

Root cause: `OnboardingViewModel.onTestAndContinue()` (the "Test & Continue" button handler on the Configure step) called `sdk.checkAuth(currentId)` for BOTH `AuthType.NONE` providers AND `config.useAnonymous=true` paths, then asserted that the result equals `AuthState.Authenticated`. For users opting INTO anonymous mode on a FORM_LOGIN tracker (RuTor with the toggle on), `checkAuth` correctly returns `Unauthenticated` — that IS the user's chosen state. The code treated it as a failure (`error = "Connection failed"`) and never advanced to the Summary step. The wait-for-"All set!" timeout fired and the user was stuck.

The C03 Challenge Test detected this only as a 60s timeout — no indication of WHY. The bug was invisible to the Sixth Law clauses 1-5 because:

- The test's primary assertion was on "All set!" appearing (✓ correct per clause 3).
- The production stack was traversed end-to-end (✓ correct per clauses 1 + 4).
- But there was no logged stack trace; the catch block called `analytics.recordNonFatal` (Crashlytics-only) and silently set the error state. The diagnostic gap was the source of the bluff: green test infrastructure + green code review + green Bluff-Audit on prior commits all missed it because nobody had run C03 end-to-end on a device with logcat tailing to see the error.

Affected files:

- `feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingViewModel.kt`

Fix:

1. Skip `sdk.checkAuth(currentId)` entirely on the anonymous branch. Anonymous = user opted out of auth = no auth state to check = not a failure when there's no session.
2. Added diagnostic `logger.d`/`logger.e` breadcrumbs on the entire onTestAndContinue flow (auth path taken, result, exception). This closes the §6.T.4-spirit gap that made the C03 bug invisible — future failures will print stack traces to logcat.
3. Added `HttpTimeout`, `UserAgent`, and `Logging` plugins to the rutracker Ktor HttpClient + 30s timeouts on the main OkHttpClient. The HTTP improvements did NOT close the C02 issue (Cloudflare-side stall — see §4.5.3b in CONTINUATION.md), but they raise the time-budget for slow networks and move the failure mode from `SocketTimeoutException` to a more diagnosable signal.
4. Bumped Challenge02 test's wait for "All set!" from 30s to 90s to align with realistic real-network round-trip times after the timeout fixes.

Verification test/challenge:

- C03 (`Challenge03AnonymousSearchOnRuTorTest`) — PASS on CZ_API34_Phone API 34 (live emulator at localhost:5555), 2026-05-12 09:25:14 to 09:25:23, total ~9.7s. Evidence at `.lava-ci-evidence/Lava-Android-1.2.13-1033/post-mortem/logcat-c03-fix.txt` shows `anon path: switchTracker(rutor) → test ok: advance to next/Summary → Perform Finish`.
- Falsifiability rehearsal: re-introducing the broken `if (result != null && result != AuthState.Authenticated)` check in the anonymous branch causes C03 to time out at 60s with the Configure screen still visible (the pre-fix failure mode). Verified through the diagnostic logcat: `checkAuth(rutor) result=Unauthenticated` → error path triggered → no advance.

Fix commit: *(this commit)*

Forensic anchor: Phase 1 systematic-debugging session 2026-05-12. Operator picked option C ("investigate C02/C03 properly") + invoked the anti-bluff mandate. Matrix runs on CZ_API34_Phone surfaced the 60s timeout; live-emulator instrumentation captured the `anon path: checkAuth(rutor) result=Unauthenticated` line that identified the bug.

2026-05-12 — onboarding-action-logger-leaks-credentials

Root cause: Discovered during the C03 investigation above — `OnboardingViewModel.perform()` logged the action via `logger.d { "Perform $action" }`. For sealed-class subtypes

`OnboardingAction.UsernameChanged(value: String)` and `OnboardingAction.PasswordChanged(value: String)`, Kotlin's auto-generated `toString` includes the value, so logcat (and any analytics breadcrumb that picks it up) prints the operator's real RuTracker username + password in plain text. Severity: §6.H credential inviolability concern — although `.env` is gitignored and the values never reach disk via a log file in the normal user flow, on a device with `adb logcat` running (or with `logcat` being collected by a 3rd party app, or shared in a bug report) this is a leak surface.

Affected files:

- `feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingViewModel.kt`

Fix: changed the log expression from `logger.d { "Perform $action" }` to `logger.d { "Perform ${action::class.simpleName}" }` — prints only the action type name, never any of its values.

Verification test/challenge: logcat tail during a re-run of C02 shows `Perform UsernameChanged` (no value) and `Perform PasswordChanged` (no value).

Fix commit: *(same commit as the c03-anonymous-toggle-checkauth-bluff entry above)*

Forensic anchor: logcat captures during the C02/C03 diag run on 2026-05-12 surfaced the bug. Pre-fix logcats are gitignored under `.lava-ci-evidence/**/post-mortem/` to prevent accidental commit of the captured credentials; only matrix attestations + gradle.logs + JUnit XML test-reports are committed.

2026-05-12 — rutracker-cloudflare-mitigation + cookie-selection-bug

Root cause 1 (Cloudflare anti-bot stall): `provideRuTrackerHttpClient` in `:core:tracker:client` constructed a `Ktor + OkHttp` HttpClient with no cookies, no Accept-Language, no Accept, no Accept-Encoding, and no explicit User-Agent — i.e. a client whose HTTP/2 header shape is a flag-raising fingerprint to Cloudflare's anti-bot. A bare POST to `rutracker.org/forum/login.php` was accepted by the TCP+TLS layer but never received a response body — verified via Ktor `Logging` plugin output. Host-side curl with normal browser-like headers returns HTTP/2 200 in <1s for the same URL.

Root cause 2 (cookie selection picked wrong cookie as session token): `RuTrackerInnerApiImpl.login()` previously extracted `token = cookies.firstOrNull { !it.contains("bb_ssl") }` from the final-hop `Set-Cookie` headers. When Cloudflare in front of

rutracker started emitting `cf_clearance=…` on every response, this filter started selecting `cf_clearance` as the "rutracker session token". The subsequent `mainPage(token)` GET then sent `Cookie: cf_clearance=…` without the actual `bb_data` session token, so rutracker returned the guest page and `parseUserId` couldn't find the logged-in element.

Affected files:

- `core/tracker/client/src/main/kotlin/lava/tracker/client/di/TrackerClientModule.kt`
- `core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/impl/RuTrackerInnerApiImpl.kt`

Fix:

1. Installed `HttpCookies` plugin (`AcceptAllCookiesStorage`) on the rutracker HttpClient.
2. Added browser-class `defaultRequest` headers (Accept, Accept-Language, Accept-Encoding) + Chrome 124 / Android 14 / Pixel 8 User-Agent.
3. Added a `runCatching { httpClient.get(Index).bodyAsText() }` pre-flight inside `RuTrackerInnerApiImpl.login()` so the `cf_clearance` cookie lands in the cookie jar before the POST.
4. Tightened the post-login token extraction to match by name prefix (`bb_data` / `bb_session` / `bb_login`) instead of the fragile `!contains("bb_ssl")` negation.

Verification test/challenge: Ktor wire-log on C02 re-run shows

- 10:16:58 GET `/forum/index.php` → 200 (~1s) — pre-flight succeeds, cookies stored
- 10:16:59 POST `/forum/login.php` → 302 (was: 60s timeout)
- 10:16:59 GET `/forum/index.php` (auto-redirect) → 200 (~1.3s) — logged-in page returned

The Cloudflare stall is fully resolved by this commit. **C02 itself does NOT yet pass end-to-end** because `GetCurrentProfileUseCase.parseUserId` throws on the post-login page — `#logged-in-username` element isn't in the served HTML (selector stale or mobile-shaped variant). That's a separate domain-archaeology task documented in `docs/CONTINUATION.md §4.5.3c`.

Fix commit: *(this commit)*

Forensic anchor: continued Phase 1 systematic-debugging session 2026-05-12 after operator's "continue" directive. C03 fix landed in `4d27c07`; the same session investigated whether the residual C02 failure was Lava-fixable. CF mitigation succeeded; the remaining parser failure surfaced a separate bug that's deferred to a follow-up SP for rutracker HTML parser refresh.

2026-05-12 — c16-stale-assumption-bluff + parser-selector-fallback + firebase-test-improvement

These three changes are landed together as the anti-bluff-audit response to the operator's SIXTEENTH §6.L invocation on 2026-05-12.

Root cause 1 (Challenge16 stale-assumption bluff): `Challenge16ApiSupportedFilterTest` asserted "Internet Archive must NOT appear in the onboarding provider list" because at the time of writing (Phase 12 α-hotfix) `ArchiveOrgDescriptor.apiSupported` was `false`. Phase 2b (Lava-API per-provider routing) later flipped that to `true` without updating the test. The test continued to pass green because its `waitUntil` clause accepted the Welcome screen — where no provider list renders — making "Internet Archive absent" trivially true. Textbook §6.L bluff: green-light while the actual behavior was the OPPOSITE of the claim.

Root cause 2 (`GetCurrentProfileUseCase.parseUserId` brittle selector): The parser used a single Jsoup selector `#logged-in-username` to extract the rutracker user-id post-login. With the Cloudflare anti-bot mitigation landing in commit `f7d0a62`, the login POST now reaches a 302→200 chain successfully — but the post-login page rutracker.org serves to the mitigated client doesn't contain `#logged-in-username` (selector stale or mobile-shaped variant). Single-selector parsers fail catastrophically when the served HTML changes; multi-selector fallback gives graceful degradation.

Root cause 3 (`FirebaseAnalyticsTrackerTest` verify-only assertions): Two tests in `FirebaseAnalyticsTrackerTest` used `verify { mock.foo() }` as the SOLE assertion — Forbidden Test Pattern per §6.L clause 4 ("Verification-only assertions"). Call-counting is permitted as a secondary signal, never the primary one.

Affected files:

- `app/src/androidTest/kotlin/lava/app/challenges/Challenge16ApiSupportedFilterTest.kt`
- `core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/domain/GetCurrentProfileUseCase.kt`
- `app/src/test/kotlin/digital/vasic/lava/client/firebase/FirebaseAnalyticsTrackerTest.kt`
- `CLAUDE.md`, `AGENTS.md`, `lava-api-go/CLAUDE.md` (§6.L count + 16th invocation forensic anchor)
- `docs/CONTINUATION.md` (audit findings + verification log)

Fix:

1. C16 rewritten to navigate to "Pick your providers" and assert that all 4 verified+apiSupported providers (RuTracker.org, RuTor.info, Internet Archive, Project Gutenberg) actually render in the list. Falsifiability rehearsal documented in the KDoc: setting `archiveorg apiSupported=false` causes the IA assertion to fail.
2. `GetCurrentProfileUseCase` now tries 4 selectors in order before erroring: `#logged-in-username`, `a.logged-in-as-uname`, `.menu-userctrl a[href*='profile.php?u=']`, `a[href*='profile.php?u=']`. Error message updated to "logged-in user-id not found — page may be guest, or selectors stale" so future agents have a clearer diagnostic anchor.
3. `FirebaseAnalyticsTrackerTest`'s `verify`-only tests refactored to use `mockk slot` captures with `assertEquals` on the captured String values + `assertTrue(slot.isCaptured)` as a non-zero-count gate. The captured-value equality is now the primary assertion, per §6.L clause 4.
4. Constitutional count update in `CLAUDE.md` §6.L (THIRTEEN → SIXTEEN), `AGENTS.md` §6.L (THIRTEEN → SIXTEEN), `lava-api-go/CLAUDE.md` §6.L (TEN → SIXTEEN). Forensic anchor for the 16th invocation appended to the wall.

Verification test/challenge:

- C16 rewritten: PASS on CZ_API34_Phone API 34 live emulator (2026-05-12 10:50, `BUILD SUCCESSFUL`).
- 14 of 23 Challenge Tests verified PASS on live emulator this session: C00, C01, C03, C11, C12, C13, C14, C15, C16(rewritten), C20, C21, C22 (in isolation; sweep-mode fails due to C21 back-press state leak — not a real-user bug), C23, C24.
- C02 still does not pass end-to-end (parseUserId fails even with fallback selectors). The Cloudflare-mitigation portion of C02 is fully verified working.

Fix commit: *(this commit)*

Forensic anchor: operator's SIXTEENTH §6.L invocation, 2026-05-12, verbatim: "Do EVERYTHING NOW!!! ... Make sure that all existing tests and Challenges do work in anti-bluff manner ... they MUST confirm that all tested codebase really works as expected! ... execution of tests and Challenges MUST guarantee the quality, the completition and full usability by end users of the product!"

2026-06-02 — apiengine-missing-serialization-plugin

Root cause: `:core:apiengine` (`core/apiengine/build.gradle.kts`) applied only `id("lava.android.library")` and did NOT apply the kotlinx-serialization compiler plugin (`id("lava.kotlin.serialization")`). `NativeApiEngine.start()` calls `json.encodeToString(config.toDto())` on the `@Serializable ConfigDto` (and `status()` decodes `@Serializable StatusDto`), but without the compiler plugin no serializers are generated, so the reflective lookup threw at runtime: `kotlinx.serialization.SerializationException: Serializer for class 'ConfigDto' is not found. Please ensure that class is marked as '@Serializable' and that the serialization compiler plugin is applied.` This is independent of R8 — the DEBUG build reproduces it. The on-device controller mapped the failure to Error, so the :api-app landing screen never reached "Running"; the Phase E real-device Challenges C02/C03/C04 timed out waiting for "Running". Fake-based unit tests (FakeApiEngine) passed because they bypass the real serializer — the canonical §6.J/§6.Z bluff class.

Affected files:

- `core/apiengine/build.gradle.kts` — applied `id("lava.kotlin.serialization")`; removed the now-redundant explicit `kotlinx.serialization.json` dep (the convention plugin contributes it).
- `core/apiengine/src/main/kotlin/lava/apiengine/NativeApiEngine.kt` — `ConfigDto`/`StatusDto` + `toDto()`/`toApiStatus()` changed `private` → `internal`, and the `defaultJson` companion `private` → `internal`, so the JVM regression test exercises the EXACT production serializer path. (Public `ApiEngine`/`ApiConfig`/`ApiStatus` API unchanged.)

Fix: apply the project's kotlinx-serialization convention plugin so the `@Serializable` DTOs get generated serializers.

Verification test/challenge:

- `core/apiengine/src/test/kotlin/lava/apiengine/ConfigSerializationTest.kt` (cheap JVM, §6.T.1 reproduction). RED with plugin removed: `kotlinx.serialization.SerializationException: Serializer for class 'ConfigDto' is not found ...` (4/4 FAIL); GREEN with plugin applied: `:core:apiengine:testDebugUnitTest BUILD SUCCESSFUL` (13/13).
- Real-device re-run (Pixel_8 / API35 / arm64-v8a, host-direct+HVF cold boot): serialization fix PROVEN — `grep -c SerializationException` over the post-Start logcat is **0** (was the Phase E blocker). C01 PASS. Evidence: `.lava-ci-evidence/`

phase-e-api-app/2026-06-02-challenge-green-after-serialization-fix.md.

Remaining defect (separate, out of this task's scope — reported, not fixed): The Challenge re-run surfaced a DISTINCT pre-existing native-linking defect in `lava-api-go/`: the prebuilt Go c-shared `liblavaapi.so` has no `SONAME`, so the JNI bridge `liblavaapi_jni.so` records the absolute host build path (`/Users/.../lava-api-go/build/jniLibs/arm64-v8a/liblavaapi.so`) as its `DT_NEEDED`, which the on-device linker cannot resolve → `dlopen failed` → embed-start unreachable → C02/C03/C04 still time out. Remediation (owed to `lava-api-go/`, NOT applied here): add `-ldflags="-extldflags=-Wl,-soname,liblavaapi.so"` to the `go build -buildmode=c-shared` in `lava-api-go/scripts/build-cshared.sh`. Full diagnosis in the evidence file above.

Fix commit: *(this commit)*

Forensic anchor: Phase E real-device Challenge run caught the serialization defect (`.lava-ci-evidence/phase-e-api-app/2026-06-02-phase-e-challenge-execution.md`) — the §6.J/§6.Z bluff class: JVM unit tests green, real embed-start broken for every user.

2026-06-02 — cshared-liblavaapi-missing-soname

Root cause: the prebuilt Go c-shared `liblavaapi.so` (built by `lava-api-go/scripts/build-cshared.sh` via `go build -buildmode=c-shared`) had NO `DT_SONAME`. The JNI bridge `liblavaapi_jni.so` (`lava-api-go/cmd/lavaapi-cshared/jni/CMakeLists.txt`) imports it as a CMake `IMPORTED SHARED` library via its absolute `IMPORTED_LOCATION`, so the NDK linker recorded the **absolute host build path** (`/Users/.../lava-api-go/build/jniLibs/arm64-v8a/liblavaapi.so`) as the bridge's `DT_NEEDED`. On-device, the Android dynamic linker cannot resolve a host filesystem path → `dlopen failed: library "/Users/.../liblavaapi.so" not found: needed by .../liblavaapi_jni.so` → `LavaNative.nativeStart` is unreachable → `ApiEngineController.start()` maps to Error → the :api-app Challenges C02/C03/C04 time out waiting for "Running". Proven with `llvm-readelf -d`, NOT guessed. (This is the "remaining defect" the 2026-06-02 serialization-fix entry above recorded as out-of-scope; it is now fixed.)

Affected files:

- `lava-api-go/scripts/build-cshared.sh` — added the NDK linker soname flag to the c-shared `go build`: `-ldflags="-extldflags=-Wl,-soname,liblavaapi.so"`.

Fix: give the c-shared `.so` a real `DT_SONAME` (`liblavaapi.so`, relative) so any consumer linking against it records the SONAME — not the absolute host path — in `DT_NEEDED`. The Android linker then resolves it from the APK's lib dir at `dlopen` time.

Verification (ELF-level, definitive):

- `llvm-readelf -d` of all 3 rebuilt ABIs: `DT_SONAME = liblavaapi.so` present (was absent before). `go build` EXIT 0 for arm64-v8a / x86_64 / armeabi-v7a; `LavaApiStart` present in each dynamic symbol table.
- `llvm-readelf -d` of `liblavaapi_jni.so` **inside the rebuilt `api-app-debug.apk`**: `(NEEDED) liblavaapi.so` (relative) — was the absolute host path. The APK ships both `lib/arm64-v8a/liblavaapi.so` (53 MB) + `liblavaapi_jni.so` (25 KB). `:api-app:assembleDebug :api-app:assembleDebugAndroidTest` BUILD SUCCESSFUL.
- Full evidence: `.lava-ci-evidence/phase-e-api-app/2026-06-02-soname-fix-and-containers-gate-blocker.md`.

Runtime gate status (honest, §6.Z/§6.J/§6.AG): the C01-04 runtime GREEN through the Containers submodule is OWED on a Containers `cmd/emulator-matrix --gradle-module` flag — the CLI is hardwired to `:app:connectedDebugAndroidTest` (`submodules/containers/pkg/emulator/{android,containerized}.go`) and cannot run `:api-app` Challenges; that submodule is out of this task's touch-list. The emulator boot itself WAS Containers-orchestrated (`--runner=auto`→host-direct+HVF, Pixel_8/API35 cold-booted AVD), proven by the runner's `[matrix-diag]` output. No green was faked; the wrong-module run was stopped before it could false-pass. `scripts/run-api-app-challenge-matrix.sh` (added this commit) is module-parameterised so the gate runs the moment Containers gains the flag.

Fix commit: *(this commit)*

Forensic anchor: the soname defect was caught by the Phase E real-device Challenge re-run AFTER the serialization fix (`.lava-ci-evidence/phase-e-api-app/2026-06-02-challenge-green-after-serialization-fix.md`) — a native-ELF bluff vector invisible to every JVM/host test: the libraries link + the APK assembles fine on the host, but the embed cannot `dlopen` on the user's device.

---

archiveorg search pagination halved totalPages (under-reported page count)

Discovered: 2026-06-06 (Completeness Program Phase 4, Stream B), while adding anti-bluff unit tests for the Internet Archive DTO→domain mapping.

Root cause: `SearchResponseDto.toDomain()` computed `totalPages` with page size 100 (`response.numFound / 100`), while the search request issued by `ArchiveOrgSearch` (and `ArchiveOrgBrowse`) uses `rows=50`. `toBrowseResult()` already used the correct `/50`. Because the divisor was double the real page size, `totalPages` for search results was under-reported by ~half — the search UI could not paginate into the back half of any result set with more than 50 hits (e.g. 1234 hits reported 13 pages instead of 25).

Affected files:

- `core/tracker/archiveorg/src/main/kotlin/lava/tracker/archiveorg/feature/ArchiveOrgDto.kt` — `toDomain()` divisor 100 → 50, matching the `rows=50` request and `toBrowseResult()`.

Fix: use page size 50 in `toDomain()` to derive `totalPages`, with a comment anchoring the value to the `rows=50` request in `ArchiveOrgSearch`.

Verification: `ArchiveOrgDtoTest.toDomain computes total pages by ceiling division of numFound by 50` asserts exact totalPages for 0/1/50/51/100/101/1234 hits. Falsifiability: re-introducing the `/100` divisor fails that test with the expected vs actual page-count mismatch; reverting to `/50` passes. The test matrix also covers `toBrowseResult` totalPages and the conditional metadata map.

Fix commit: *(this commit)*

---

formatSize rendered comma-decimal separators on non-US locales

Discovered: 2026-06-06 (Completeness Program Phase 3, Detekt correctness backlog).

Root cause: `core/tracker/rutracker/.../domain/Utils.kt:102` `formatSize` used `String.format("%.1f %sB", …)` without a `Locale`, so the JVM default locale applied. On any comma-decimal locale (de/ru/fr/…) a 1.5 MB torrent rendered as "1,5 MB".

Fix: `String.format(Locale.ROOT, …)`.

Verification: `FormatSizeLocaleTest` forces `Locale.GERMANY`/`ru-RU` and asserts dot output. Falsifiability: Locale-less format → 2 of 4 tests FAIL (expected "1.5 MB" but was "1,5 MB"). **Fix commit:** bc0a478d.

DownloadServiceImpl cache data race

Discovered: 2026-06-06 (Completeness Program Phase 3, Android leak/race sweep).

Root cause: `DownloadServiceImpl.cache` was a plain HashMap read on the caller's coroutine dispatcher (`downloadTorrentFile`) and written from the `DownloadManager` `BroadcastReceiver.onReceive` (main thread). Concurrent downloads touched a non-thread-safe map across two threads.

Fix: extracted `DownloadUriCache` backed by `ConcurrentHashMap` (pure-JVM, unit-testable per Fifth Law).

Verification: `DownloadUriCacheConcurrencyTest` (16 threads × 500 keys). Falsifiability: HashMap backing → test FAILS; ConcurrentHashMap → 3/3 green. **Fix commit:** 76fb0879.

ApiEngineController.start() TOCTOU double-bind

Discovered: 2026-06-06 (Completeness Program Phase 3/4).

Root cause: `ApiEngineController.start()` re-entry guard was a read-then-set on `MutableStateFlow`. The controller is a Hilt singleton driven by the ViewModel (Dispatchers.Default) and the Service (Main.immediate, ACTION_RESTART); two concurrent start() calls could both observe Stopped and both bind the embed ("address already in use").

Fix: atomic `_state.compareAndSet` guard loop + a `@VisibleForTesting` guard-checkpoint seam (Fifth Law: refactor for testability).

Verification: `ApiEngineControllerTest.concurrent start parked in the guard gap binds the embed exactly once` parks one caller in the read→set gap. Falsifiability: read-then-set → test FAILS (double-bind); compareAndSet → 8/8. **Fix commit:** 30e880c6.

TestSearchHistoryRepository.remove kept the matched row (bluff fake)

Discovered: 2026-06-06 (Completeness Program Phase 4, flagged independently by two subagent streams).

Root cause: the shared `:core:testing` fake's `remove(id)` did `filter { it.id == id }` — it KEPT the matched row and dropped the others, the inverse of the real `SearchHistoryRepositoryImpl.remove` (Room `delete(id)`). A Third-Law bluff fake.

Fix: `filterNot { it.id == id }`; added `TestSearchHistoryRepositoryTest`.

Verification: the test asserts remove(1) on [0,1,2] leaves [0,2]. Falsifiability: the shipped bug fails it (expected [0,2] but was [1]). feature/search + feature/menu stay green. **Fix commit:** *(this session)*.

On-device API embed could silently drift from the lava-api-go source (no input-tracking)

Discovered: 2026-06-06 (Completeness Program, STREAM API-SYNC).

Root cause: the `core:apiengine` Gradle `buildCshared` task declared ONLY `inputs.file(cSharedScript)` as its input. The lava-api-go SOURCE TREE that actually compiles into `liblavaapi.so` was NOT declared as a task input, so Gradle considered the task up-to-date whenever the per-ABI `.so/.h` outputs existed — even after the lava-api-go Go source changed. A developer who edited `lava-api-go/internal/...` and re-assembled the api-app got the OLD `.so`: the on-device API embed silently drifted from the current API codebase, while every other gate stayed green (a §11.4.69 / §6.J "tests pass, wrong-version feature" bluff vector). There was no mechanism asserting "the packaged embed equals the current API source".

Affected files:

- `core/apiengine/build.gradle.kts` (missing source inputs on `buildCshared`)
- `lava-api-go/scripts/build-cshared.sh` (no source-hash injection / manifest)
- `lava-api-go/internal/version/version.go`, `internal/mobile/mobile.go`
- `core/apiengine/.../ApiEngine.kt`, `NativeApiEngine.kt`

Fix (drift-prevention mechanism):

1. `scripts/compute-api-source-hash.sh` — single source of truth: a deterministic sha256 over the exact non-test `.go` files under `cmd/lavaapi-cshared` + `internal` plus `go.mod`/`go.sum` (the file set that links into the `.so`).

2. `build-cshared.sh` injects that hash into `version.SourceHash` via `-ldflags -X` (the running `.so` reports it through `mobile.Status()`), and writes the committed manifest `core/apiengine/src/main/resources/api-source.hash` on every successful build.
3. `scripts/check-api-app-sync.sh` — CI gate (wired into `scripts/ci.sh`, both `--changed-only` and `--full`): recompute the live hash, compare to the committed manifest, exit 1 loudly on mismatch.
4. `core/apiengine/build.gradle.kts` `buildCshared` now declares the lava-api-go source dirs + `go.mod`/`go.sum` as `@InputFiles`, so Gradle re-runs `build-cshared.sh` on ANY embed-source change (kills the stale-cache drift).
5. On-device proof: api-app Challenge C05 (`Challenge05ApiEmbedSourceHashMatchesTest`) starts the real embed and asserts `ApiStatus.sourceHash == BuildConfig.LAVA_API_SOURCE_HASH` (no drift), with an empty-hash hard-fail discrimination.

Verification: `lava-api-go/tests/contract/sourcehash_contract_test.go` asserts hash stability, manifest-matches-live, and falsifiability (a content edit to a tracked embed-linked `.go` changes the hash). 3/3 PASS. The gate `scripts/check-api-app-sync.sh` PASSES on the seeded manifest; falsifiability rehearsal: appending a byte to `internal/version/version.go` → gate exits 1 ("on-device API embed is STALE"); revert → exit 0. **Fix commit:** *(this session)*.

2026-06-08 — kinozal-magnet-link-declared-but-null (§6.E bluff)

Root cause: `KinozalDescriptor` declares the `MAGNET_LINK` capability, but `KinozalDownload.getMagnetLink()` returned `null` even though the magnet is parsed from the topic/search HTML — a declared-but-empty capability, i.e. a §6.E capability-honesty bluff. The `getMagnetLink` interface method is non-suspend and cannot itself fetch.

Affected files:

- `core/tracker/kinozal/src/main/kotlin/lava/tracker/kinozal/feature/KinozalDownload.kt`
- `core/tracker/kinozal/src/main/kotlin/lava/tracker/kinozal/feature/KinozalTopic.kt`
- `core/tracker/kinozal/src/main/kotlin/lava/tracker/kinozal/magnet/KinozalMagnetCache.kt` (NEW)
- `core/tracker/kinozal/src/main/kotlin/lava/tracker/kinozal/KinozalClientFactory.kt`
- `core/tracker/kinozal/src/test/kotlin/lava/tracker/kinozal/feature/KinozalDownloadTest.kt`

- `core/tracker/kinozal/src/test/kotlin/lava/tracker/kinozal/feature/KinozalTopicTest.kt`
- `core/tracker/kinozal/src/test/kotlin/lava/tracker/kinozal/KinozalClientFactoryCloneUrlTest.kt`

Fix: wire it honestly — a `@Singleton KinozalMagnetCache` populated by the real topic-view path (`KinozalTopic.getTopic`), read back synchronously by `getMagnetLink(id)`; honest `null` until a topic surfaces it. Same pattern RuTracker documents for `GetMagnetLinkUseCase`. Cache threaded through the clone-override path.

Verification test/challenge: `KinozalDownloadTest` — "getMagnetLink returns real magnet after the topic page has been viewed". Falsifiability rehearsal (per commit body): reverting `getMagnetLink` to `= null` (the original defect) FAILED with `AssertionError: magnet should be exposed after topic view (KinozalDownloadTest.kt:59)`; reverted, `core:tracker:kinozal:test` all green (forced `--rerun-tasks`).

Fix commit: `6fa31ad2`

Forensic anchor: §6.E capability-honesty audit, 2026-06-08 session.

2026-06-08 — nnmclub-magnet-link-declared-but-null (§6.E bluff)

Root cause: `NnmclubDescriptor` declares `MAGNET_LINK`, but `NnmclubDownload.getMagnetLink()` returned `null` though both `NnmclubSearchParser` and `NnmclubTopicParser` parse the genuine magnet from NNM-Club HTML — a §6.E bluff.

Affected files:

- `core/tracker/nnmclub/src/main/kotlin/lava/tracker/nnmclub/feature/NnmclubDownload.kt`
- `core/tracker/nnmclub/src/main/kotlin/lava/tracker/nnmclub/feature/NnmclubTopic.kt`
- `core/tracker/nnmclub/src/main/kotlin/lava/tracker/nnmclub/feature/NnmclubSearch.kt`
- `core/tracker/nnmclub/src/main/kotlin/lava/tracker/nnmclub/http/NnmclubMagnetCache.kt` (NEW)
- `core/tracker/nnmclub/src/main/kotlin/lava/tracker/nnmclub/NnmclubClientFactory.kt`
- `core/tracker/nnmclub/src/test/kotlin/lava/tracker/nnmclub/feature/NnmclubMagnetExposureTest.kt` (NEW)
- `core/tracker/nnmclub/src/test/kotlin/lava/tracker/nnmclub/feature/NnmclubDownloadTest.kt`

- core/tracker/nnmclub/src/test/kotlin/lava/tracker/nnmclub/feature/NnmclubSearchTest.kt
- core/tracker/nnmclub/src/test/kotlin/lava/tracker/nnmclub/feature/NnmclubTopicTest.kt
- core/tracker/nnmclub/src/test/kotlin/lava/tracker/nnmclub/NnmclubClientTest.kt
- core/tracker/nnmclub/src/test/kotlin/lava/tracker/nnmclub/NnmclubClientFactoryCloneUrlTest.kt
- core/tracker/nnmclub/src/test/resources/fixtures/nnmclub/search/search-with-magnet-2026-06-08.html (NEW fixture)

Fix: wire it honestly — a `@Singleton NnmclubMagnetCache` populated by the real topic-view (`NnmclubTopic.getTopic`) and search-row (`NnmclubSearch.search`) paths, read back by `getMagnetLink(id)`; honest `null` when nothing has surfaced it. Cache threaded through the clone-override path. Adds a real fixture (`search-with-magnet-2026-06-08.html`) containing a magnet row.

Verification test/challenge: `NnmclubMagnetExposureTest` (`lava.tracker.nnmclub.feature`). Falsifiability rehearsal (per commit body): reverting `getMagnetLink` to `return null` (the historical bluff) FAILED 2 of 3 — "exposes the parsed magnet after the topic page surfaces it" + "after a search row surfaces it" — `AssertionError` at `NnmclubMagnetExposureTest.kt:50` and `:82`; reverted, `core:tracker:nnmclub:test` 27 tests / 0 failures (forced `--rerun-tasks`).

Fix commit: `13703bc8`

Forensic anchor: §6.E capability-honesty audit, 2026-06-08 session.

2026-06-08 — rutor-magnet-link-declared-but-null + bluff-codifying test (§6.E)

Root cause: `RuTorDescriptor` declares `MAGNET_LINK` but `RuTorDownload.getMagnetLink()` returned an unconditional `null` though `RuTorTopicParser` + `RuTorSearchParser` parse the genuine magnet from RuTor HTML — a §6.E bluff. Worse, a pre-existing test "rutor has no synchronous magnet (Capability Honesty)" CODIFIED the bluff as intentional.

Affected files:

- core/tracker/rutor/.../feature/RuTorDownload.kt
- core/tracker/rutor/.../feature/RuTorTopic.kt
- core/tracker/rutor/.../feature/RuTorSearch.kt
- core/tracker/rutor/.../magnet/RuTorMagnetCache.kt (NEW)

- `core/tracker/rutor/.../RuTorClientFactory.kt`
- `core/tracker/rutor/.../feature/RuTorMagnetExposureTest.kt` (NEW)
- `core/tracker/rutor/.../feature/RuTorDownloadTest.kt`
- `core/tracker/rutor/.../feature/RuTorSearchTest.kt`
- `core/tracker/rutor/.../feature/RuTorTopicTest.kt`
- `core/tracker/rutor/.../RuTorClientTest.kt`
- `core/tracker/rutor/.../RuTorClientFactoryCloneUrlTest.kt`
- `.gitmodules` + `submodules/{doc_processor,llm_orchestrator,llm_provider,llms_verifier,vision_engine}` (the 5 HelixQA dep submodule gitlinks landed in this same commit; see the HelixQA pin entry below)

> UNCONFIRMED: the exact `core/tracker/rutor` source-root prefix is abbreviated above; `git show --stat 6d87019d` reports the file basenames but the path column is truncated in the stat output. The basenames + `feature/` / `magnet/` subdirs are confirmed.

Fix: wire it honestly via a `@Singleton RuTorMagnetCache` populated by the real topic-view + search-row paths, read back by `getMagnetLink(id)`; honest `null` on miss. Threaded through the clone-override path. The bluff-codifying test was replaced with honest-absence semantics.

Verification test/challenge: `RuTorMagnetExposureTest`. Falsifiability rehearsal (per commit body): reverting `getMagnetLink` to `return null` (the historical bluff) FAILED 2 of 3 — "exposes the parsed magnet after the topic page surfaces it" + "after a search row surfaces it" — `AssertionError` at `RuTorMagnetExposureTest.kt:57` and `:97`; reverted, `core:tracker:rutor:test` 76 tests / 0 failures; `RuTorMagnetExposureTest` 3/0/0 (forced `--rerun-tasks`).

Fix commit: `6d87019d`

Forensic anchor: §6.E capability-honesty audit, 2026-06-08 session — this one is the sharpest §6.J case of the four because a passing test had been encoding the bluff as the intended contract.

2026-06-08 — gutenberg-dishonest-torrent-download-capability (§6.E)

Root cause: `GutenbergDescriptor` declared `TORRENT_DOWNLOAD` but the provider serves HTTP e-books (Gutendex EPUB/text/HTML), not `.torrent` files — a §6.E capability-honesty bluff. The downloaded artifact is not a bencoded `.torrent`.

Affected files:

- `core/tracker/gutenberg/src/main/kotlin/lava/tracker/gutenberg/GutenbergDescriptor.kt`
- `core/tracker/gutenberg/src/main/kotlin/lava/tracker/gutenberg/GutenbergClient.kt`
- `core/tracker/gutenberg/src/test/kotlin/lava/tracker/gutenberg/GutenbergCapabilityHonestyTest.kt` (NEW)
- `core/tracker/gutenberg/src/test/kotlin/lava/tracker/gutenberg/GutenbergClientTest.kt`
- `core/tracker/gutenberg/src/test/kotlin/lava/tracker/gutenberg/GutenbergDescriptorTest.kt`

Fix: mirror archiveorg's established honest pattern — drop `TORRENT_DOWNLOAD` from the descriptor; `GutenbergClient.getFeature( DownloadableTracker::class)` returns `null` explicitly (the `GutenbergDownload` impl stays wired but unexposed via the torrent surface). No enum/api change. Stale tests that encoded the bluff were removed.

Verification test/challenge: `GutenbergCapabilityHonestyTest` pins the contract. Falsifiability rehearsal (per commit body): re-declaring `TORRENT_DOWNLOAD` + re-gating `getFeature` so the EPUB-serving impl is again exposed via the torrent-download surface FAILED — "download capability is honest about the artifact it produces" — `AssertionError: TORRENT_DOWNLOAD declared but downloaded artifact is not a bencoded .torrent (first byte was 'e', expected 'd') (GutenbergCapabilityHonestyTest.kt:91);` reverted, independently re-verified `GutenbergCapabilityHonestyTest` 2/0/0.

Fix commit: `b197f96d`

Forensic anchor: §6.E capability-honesty audit, 2026-06-08 session.

2026-06-08 — helixqa-broken-pin-gomod-conflict-markers (broke lava-api-go build)

Root cause: HelixQA's `go.mod` `replace` directives require 8 sibling own-org (`digital.vasic.*`) Go modules; 3 were present (challenges/containers/security), 5 were MISSING, so the full `helixqa` binary could not build. The broken pinned HelixQA `go.mod` also carried unresolved merge-conflict markers (at lines 124 / 131 / 138 per the commit body), which broke `lava-api-go`'s Go build.

Affected files:

- `submodules/helixqa` (pin bump `5112906` → `dd3cf1d`)
- `docs/qa/helixqa-dependency-submodules.md` (NEW doc)
- `docs/CONTINUATION.md` (§0 + §3 sync)
- (paired commit `6d87019d` adds the 5 dep submodule gitlinks `submodules/{doc_processor,llm_orchestrator,llm_provider,llms_verifier,vision_engine}` from `vasic-digital/{DocProcessor,LLMOrchestrator,LLMProvider,LLMsVerifier,VisionEngine}`)

> UNCONFIRMED: the conflict-marker line numbers 124/131/138 are quoted from the commit body, not independently re-diffed in this audit. The pin SHAs `5112906 → dd3cf1d` and the `submodules/helixqa` change are confirmed by `git show --stat dd72669b`.

Fix: bump the helixqa pin `5112906 → dd3cf1d` (clean `go.mod`); add the 5 missing dep submodules (local paths lowercase snake_case per §11.4.29 matching HelixQA's `../doc_processor` replace targets; URLs use the real CamelCase repo names per the must-not-break-technology carve-out).

Verification: physical proof recorded in the commit body — `go build ./cmd/helixqa` exit 0 → 29.7 MB binary; `helixqa version` → v0.2.0; and `lava-api-go` builds (exit 0).

Fix commit: `dd72669b` (pin bump + doc) — the 5 dep submodule gitlinks landed in companion commit `6d87019d`.

Forensic anchor: operator directive ("add all missing dependency Submodules from vasic-digital/HelixDevelopment") + §11.4.27/§11.4.28 (own-org deps reachable from root). OWED (tracked in CONTINUATION): §6.W GitLab mirrors for the 5 new submodules; §11.4.29 upstream CamelCase→snake_case repo rename.

2026-06-08 — nav-compose-2.9.0-test-teardown-lifecycle-race

Root cause (confirmed, stack-trace-quoted in the systematic-debugging investigation): androidx-navigation-compose 2.9.0's single-top `NavBackStackEntry` lifecycle reaches `CREATED` asynchronously; the Espresso/Compose test-runner's immediate `Activity.performDestroy` then tries `INITIALIZED → DESTROYED` in one step → `IllegalStateException "State must be at least 'CREATED' to be moved to 'DESTROYED'"`. 2.9.1 release note: "Fixed an issue that caused NavEntries instantiated using single

top to never go beyond CREATED in their Lifecycle.State" ([I043ba], b/421095236) — matches exactly (Lava uses `launchSingleTop=true`).

User impact: NONE — real users get intervening lifecycle pauses. It is a synthetic test-teardown artifact that had forced Challenge tests C04–C08 + deep C11 to be gutted to shallow versions. Blast radius LOW: single direct consumer (`core/navigation`); `-Xcontext-receivers` is orthogonal (a Kotlin compiler feature, not the nav lib).

Affected files:

- `gradle/libs.versions.toml` (pin `2.9.0 → 2.9.1`)

Fix: bump `androidx-navigation-compose` `2.9.0 → 2.9.1`.

Verification: `./gradlew :core:navigation:compileDebugKotlin` → BUILD SUCCESSFUL with 2.9.1.

> UNCONFIRMED / OWED (per commit body): the race-fix confirmation (restoring the deep C04–C08/C11 Challenges + proving the teardown crash is gone) is device-gated — requires the Genymotion VM / a real device, OWED when a device is booted. Not yet verified on-device in this session.

Fix commit: `7e6e7bcb`

Forensic anchor: systematic-debugging investigation of the C04–C08/C11 shallow-test regression, 2026-06-08 session.

---

§6.E — RuTracker `MAGNET_LINK` declared but `getMagnetLink` returned null (2026-06-08)

Class: §6.E capability-honesty bluff (declared-but-empty capability).

Symptom: `RuTrackerDescriptor` declares `TrackerCapability.MAGNET_LINK` (`RuTrackerDescriptor.kt:36`) and `getFeature(DownloadableTracker)` resolves to a non-null `RuTrackerDownload`, but the only reachable `getMagnetLink` impl (`GetMagnetLinkUseCase`) returned `null` unconditionally for EVERY id on the real stack — even though RuTracker genuinely parses the magnet (`TopicMapper.kt:121`). A user tapping "Magnet" on a RuTracker topic could never get the magnet the app had already parsed. Identical to the bluff RuTor closed with `RuTorMagnetCache`.

Root cause: `GetMagnetLinkUseCase.invoke(id) = null` stub; no path carried the parsed `TorrentItem.magnetUri` from the topic/search fetch to the synchronous `DownloadableTracker.getMagnetLink`. Audit: `docs/qa/magnet-label-honesty-audit-2026-06-08.md` (W4a).

Affected files:

- `core/tracker/rutracker/.../magnet/RuTrackerMagnetCache.kt` (new — RuTor pattern)
- `core/tracker/rutracker/.../domain/GetMagnetLinkUseCase.kt` (reads the cache)
- `core/tracker/rutracker/.../feature/RuTrackerTopic.kt` (populates on getTopic)
- `core/tracker/rutracker/.../feature/RuTrackerSearch.kt` (populates per result row)
- `core/tracker/rutracker/.../RuTrackerSubgraphBuilder.kt` (shared cache, clone path)
- `core/tracker/client/.../di/TrackerClientModule.kt` (Hilt @Provides reads the cache)

Fix: adopt the RuTor magnet-cache pattern — a `@Singleton` process-lifetime `RuTrackerMagnetCache` populated by `RuTrackerTopic.getTopic` / `RuTrackerSearch.search` from the already-mapped `magnetUri`, read by `GetMagnetLinkUseCase`. Honest null preserved for ids never surfaced.

Verification (§6.T.1 reproduction-before-fix):

- RED (pre-fix, against the stub): `RuTrackerMagnetExposureTest > getMagnetLink exposes the magnet the topic mapper surfaced` FAILED — "§6.E BLUFF: ... getMagnetLink returned null".
- Mutation rehearsal (post-fix, `invoke → null`): same test FAILED again (exit 1).
- GREEN (fix): `RuTrackerMagnetExposureTest` 2/2 PASS; `:core:tracker:client:testDebugUnitTest` BUILD SUCCESSFUL (Hilt graph valid; enumeration gate 5/5 + 8/8).

Forensic anchor: SA4 §6.E honesty audit, 2026-06-08 parallel-fleet session. Full-loop (topic fetch → cache → download) covered on-device by the restored C05/C06 Challenges (device run owed).

---

§6.Q/C37 — CrossTrackerFallbackModal dead-ended at the screen layer (2026-06-08)

Class: dead-UI (capability present in code, unreachable by users — the §6.Q/C37 class).

Symptom: `feature/search_result` has a full cross-tracker-fallback feature — `CrossTrackerFallbackModal` composable + `SearchResultViewModel.proposeFallback/ onFallbackAccept/ onFallbackDismiss` + `SearchResultAction.FallbackAccept/ FallbackDismiss` + `SearchResultState.crossTrackerFallback` — BUT `SearchResultScreen` never read `state.crossTrackerFallback` and never called `CrossTrackerFallbackModal(...)`. A real user whose search failed on one tracker could never see the "Try " offer. Found by the SA5 Challenge restoration audit (C07/C08 had to be written as contract guards, not the real flow, because the real flow was unreachable).

Affected files:

- `feature/search_result/.../SearchResultScreen.kt` (renders the modal)
- `app/src/androidTest/.../challenges/ Challenge07CrossTrackerFallbackAcceptTest.kt`
- `app/src/androidTest/.../challenges/ Challenge08CrossTrackerFallbackDismissTest.kt`

Fix: in `SearchResultScreen`, render `CrossTrackerFallbackModal(failedTracker, proposedTracker, onAccept = { onAction(FallbackAccept) }, onDismiss = { onAction( FallbackDismiss) })` when `state.crossTrackerFallback != null` (display names via the existing `state.providerDisplayNames`). C07/C08 upgraded from contract guards to real rendered-modal Compose tests (C30 pattern): render → assert "RuTracker is unavailable" + "Try RuTor"/"Cancel" → tap → assert the callback fires.

Verification: `:feature:search_result:compileDebugKotlin` + `:app:compileDebugAndroidTestKotlin` BUILD SUCCESSFUL. §6.J falsifiability (KDoc): delete the `state.crossTrackerFallback?.let{}` block → dead-end returns; or change the confirm/dismiss label → the rendered test's `onNodeWithText` finds no node → assert fails. Device-run of C07/C08 owed on the VM (network-independent).

Forensic anchor: SA5 Challenge-restoration finding, 2026-06-08 parallel-fleet session.

---

§6.J DEVICE-GATE CORRECTION — nav-compose 2.9.1 does NOT fully fix the NavBackStackEntry teardown crash (2026-06-08)

Correcting the 2.9.0→2.9.1 entry (commit `7e6e7bcb`). That bump claimed to fix the test-teardown NavBackStackEntry lifecycle race and explicitly marked the device confirmation OWED. Device confirmation is now done — on the Genymotion Pixel 9 VM (API 35) — and it **FALSIFIES the claim** for the `search_input` route: `Challenge11ArchiveOrgAnonymousSearchTest` crashes the app at `MainActivity` destroy with `IllegalStateException: State must be at least 'CREATED' to be moved to 'DESTROYED'` on the `search/search_input?...` NavBackStackEntry (`LifecycleRegistry.kt:92`). 2.9.1 was necessary but NOT sufficient.

Status: OPEN. C00/C01/C07/C08 PASS on the VM (6/7); C11 RED. NOT bluffed green. Incident: `.lava-ci-evidence/sixth-law-incidents/2026-06-08-navbackstackentry-teardown-crash-2.9.1-incomplete.json`. Evidence: `.lava-ci-evidence/genymotion/9e7505b3-fleet-run/`.

Next: dedicated systematic-debugging cycle — evaluate a newer androidx-navigation OR a NavHost/teardown lifecycle guard, then device re-verify.

api-source.hash manifest staleness — §6.A contract test RED→GREEN (2026-06-08)

Commit `7ce9c2b2` edited two embed-linked lava-api-go sources (`internal/cache/cache.go` + `internal/storage/sqlite.go`) without regenerating the API↔embed sync manifest, so `TestSourceHash_ManifestMatchesLive` (`lava-api-go/tests/contract`) failed: committed `95c36d1773362cee2b90db398f2e2827586fef83071ba3feb12ce51e02e12b9d` ! = live `70ebb53a1a9c3365f651ae7ae651dd4404d58976b97b3b2589db39f4d7752470`. Root cause: the manifest at `core/apiengine/src/main/resources/api-source.hash` was last written at `e153d7c0` (2026-06-06), pre-dating `7ce9c2b2`. The contract test correctly caught real source/embed drift — exactly its §6.A purpose.

Fix: regenerated `api-source.hash` to the live `scripts/compute-api-source-hash.sh` value (the canonical mechanism `build-cshared.sh` uses). NOT a test edit — the regeneration the test's own error message prescribes. Verified: `go test ./tests/contract -run`

`TestSourceHash` → 3/3 PASS (incl. the falsifiability sibling … `ContentEditChangesHash`); full `go test ./...` → exit 0 (39 packages ok, real Postgres-in-podman e2e ran).

YTS curated provider unreachable — yts.mx domain rotated out of DNS (2026-06-13)

Symptom: the YTS curated provider (shipped 2026-06-12, commit `462a0308`) passed all fixture unit tests but its LIVE call failed: `live Search: yts: provider: unknown error`. Surfaced by a main-stream §6.J re-verification (running the `-tags realtrackers` live tests instead of trusting the prior commit-body claim).

Root cause: the provider hardcoded a single base URL `https://yts.mx`. As of 2026-06-13 `yts.mx` has NO A record on public DNS (confirmed `dig @1.1.1.1` and `@8.8.8.8` both return `<none>`) — YTS rotated its domain since the provider shipped. The fixture parser was correct (unit tests green) while the feature was broken for real users (§6.G/§6.L fixture-green / feature-broken). The current live canonical host is `yts.bz` (yts.lt / yts.am 301-redirect to it).

Affected files: `lava-api-go/internal/provider/curated/yts/{client.go,provider.go,live_realtrackers_test.go,yts_test.go}`.

Fix: mirror failover — `Client` now holds a list of base URLs (`DefaultBaseURLs` = yts.mx, yts.bz, yts.lt, yts.am) tried in order; the first successful answer wins; each attempt is bounded by `perAttemptTimeout` (8s) so a dead/hanging mirror cannot stall the search. `NewClient(single)` preserved as the httptest seam; production `New()` uses `NewClientWithMirrors(DefaultBaseURLs)`. The live test now queries a real movie TITLE (`interstellar`; `1080p` matches no movie and legitimately returns 0) across the mirror list.

Verification: new `TestSearch_FailsOverToHealthyMirror` (falsifiable: break the failover loop → `Search across [dead, healthy] mirrors should fail over, got error: yts: HTTP 503`; reverted) + `TestSearch_AllMirrorsDownSurfacesError`. Live `-tags realtrackers` PASS in 0.559s (fast failover past dead yts.mx to a live mirror, real magnets). Full `go test ./...` → 0 FAIL; embed source-hash regenerated `0d7da603`; `check-api-app-sync.sh` GREEN. Evidence: `.lava-ci-evidence/bluff-hunt/2026-06-13-yts-domain-failover.json`.

Follow-up: TPB (apibay.org) + Torrents-CSV (torrents-csv.com) still hardcode a single domain — same rotation-risk class; mirror-failover hardening noted as a follow-up.

Crashlytics remediation sweep — 6 open issues (2026-06-13)

Operator directive: investigate ALL open Crashlytics issues, systematic-debug, fix/improve, cover with falsifiable validation tests + §6.O closure logs. Real data pulled from Firebase (project `lava-vasic-digital`, release app). Per-issue:

- `7df61fdb` **NON_FATAL JobCancellationException (8 events, 1.2.21) — FIXED.** Catch-site rethrow (defense-in-depth over the prior sink-level filter). New `Throwable.rethrowIfCancellation()` in `core/common/.../analytics/CancellationRethrow.kt` called first in every coroutine catch/onFailure that records a non-fatal across 8 ViewModels (onboarding, login, menu, search_input, search_result, topic, bookmarks, favorites). Root cause: broad `catch (e: Exception)` swallowed Kotlin's structured-concurrency `CancellationException` — both telemetry pollution AND broken cooperative cancellation. Test: `core/common/.../CancellationRethrowTest.kt` (5 tests). Bluff-Audit: no-op the rethrow → 3 tests RED at CancellationRethrowTest.kt:75; reverted. Closure: `.lava-ci-evidence/crashlytics-resolved/2026-06-13-jobcancellation-catch-site-rethrow.md`.

- `40a62f97` **FATAL painterResource layer-list (1.2.19) — VERIFIED already covered.** `LavaIconsAppIconColorRegressionTest` + closure `2026-05-14-welcome-layerlist-painter-crash.md` already in place. No change.

- `39469d3b` **FATAL okhttp schemeless URL "djdnjd" (1.2.21) — FIXED (owed regression test added).** The prior closure admitted the use-case test was "owed in a follow-up"; this is it. New `ProbeMirrorUseCaseTest.schemeless URL returns Unreachable instead of crashing` (real `ProbeMirrorUseCase` + real OkHttpClient). Bluff-Audit: change the `catch (IllegalArgumentException)` to a non-matching type → test RED with the leaked `IllegalArgumentException` at :80; reverted. Closure: `.lava-ci-evidence/crashlytics-resolved/2026-06-13-probemirror-schemeless-url-regression.md`.

- `042b9b61` **NON_FATAL CertPath trust-anchor (1.3.3) — VERIFIED already covered.** Fixed at 1.3.4 (commit `0deb54e7`); `ProviderCatalogRepositoryTest` uses a self-signed-TLS MockWebServer; closure `2026-06-12-provider-catalog-fetch-tls.md` present. No change.

- `6519b490` **NON_FATAL rutracker parseUserId "user-id not found" (1.2.22) — WORKING-AS-DESIGNED (no production change).** New `GetCurrentProfileUseCaseUserIdTest` proves the 4 production selectors DO parse a logged-in page

(not stale) and the non-fatal fires only on a genuine guest/expired page. The logged-in test is the future staleness canary. Bluff-Audit: reduce selectors to a non-matching one → canary RED at :100; reverted. Closure: `.lava-ci-evidence/crashlytics-resolved/2026-06-13-rutracker-parseuserid-guest-page.md`.

- **`3937b7f0` NON_FATAL SSE "Unable to resolve host lava-api.local" (1.3.0) — IMPROVED.** `SearchResultViewModel.applySseError` now classifies connectivity-class reasons (host-resolve / refused / timeout) as a lower-severity `recordWarning("sse_endpoint_unreachable")` instead of a crash-feed non-fatal; genuine backend errors stay non-fatals. User-visible Error + Retry UX unchanged. New `String.isConnectivityFailure()` helper. Test: `SearchResultSseConnectivityTelemetryTest` (2 tests). Bluff-Audit: force the classifier to `if (false)` → host-resolve test RED; reverted. Closure: `.lava-ci-evidence/crashlytics-resolved/2026-06-13-sse-host-resolve-telemetry-severity.md`.

Affected production files: `core/common/.../analytics/CancellationRethrow.kt` (new); 8 feature ViewModels (rethrow calls); `core/common/build.gradle.kts` (coroutines-test dep); `feature/search_result/.../SearchResultViewModel.kt` (SSE severity classifier). `ProbeMirrorUseCase.kt` + rutracker `GetCurrentProfileUseCase.kt` UNCHANGED (already-correct fixes; tests added).

§6.AC/§6.O — JobCancellation non-fatal noise: download catch-site rethrow (2026-06-13)

Crashlytics: `7df61fdba64f9928b067624d6db395ca` (NON_FATAL, `kotlinx.coroutines.JobCancellationException` — "StandaloneCoroutine was cancelled", 8 events / 1 user / first=last 1.2.21; not recurring on 1.3.x).

Root cause. `DownloadServiceImpl.downloadHttpFile` (`core/downloads`) — a `suspend fun` — wrapped its write path in a broad `catch (t: Throwable)` that called `analytics.recordNonFatal(t, ...)`. When the download is abandoned mid-write (calling scope cancelled — user left the screen / ViewModel cleared), `CancellationException` was swallowed AND recorded as a non-fatal: false telemetry (issue `7df61fdb` dashboard noise) plus broken cooperative cancellation (the catch returned `null` instead of letting the cancellation propagate). A grep of every telemetry-recording `catch` across `core/ feature/ app/` main sources confirmed this was the ONLY remaining offending site — all 8 feature ViewModels + `LavaApplication` already call `rethrowIfCancellation()`.

Fix. `t.rethrowIfCancellation()` (the existing shared `lava.common.analytics` helper — reused, not re-created) is now the FIRST statement of the `downloadHttpFile` catch, before `recordNonFatal`.

Verification test. `core/downloads/src/test/.../DownloadServiceCancellationTest.kt` (new) — `cancellation during http write is rethrown and never recorded as non-fatal`. Real `DownloadServiceImpl` + recording `AnalyticsTracker`; only the outermost Android boundary (`Context` / the `Environment` static the write path uses) is faked to throw `CancellationException` mid-write. Asserts the cancellation PROPAGATES and the recorded-non-fatal count is `0`. Bluff-Audit: delete the rethrow line → `AssertionError: cancellation must propagate, not be swallowed into a null return` (`DownloadServiceCancellationTest.kt:87`); reverted → BUILD SUCCESSFUL.

Affected files: `core/downloads/.../DownloadServiceImpl.kt` (rethrow), `core/downloads/build.gradle.kts` (mockk + coroutines-test test deps + `returnDefaultValues`), `core/downloads/.../DownloadServiceCancellationTest.kt` (new). Closure log: `.lava-ci-evidence/crashlytics-resolved/2026-06-13-cancellation-noise-7df61fdb.md`. Queued for the 1.3.7 release cycle (version files NOT touched).

chosen online server appears TWICE in Settings → Server list (2026-06-14)

Symptom (operator-reported): "When we open settings Server list, the chosen online server appears TWICE in the list." The user onboarded with an online/cloud API endpoint; in Settings → the server/connection list, that chosen endpoint is shown duplicated.

Root cause: `core/data/src/main/kotlin/lava/data/converters/Endpoint.kt:69` builds the Room PRIMARY KEY id for an `Endpoint.GoApi` as `GoApi(${packHost()})`, and `packHost()` (`Endpoint.kt:52-60`) appends the additive `key`/`platform`/`storage` fields after a `#` sentinel. The SAME physical server (same host:port) reaches the persisted list via TWO paths that differ only in those additive fields:

- the cloud "Add server" flow (`feature/onboarding/.../OnboardingViewModel.kt:179 onAddCloudApi` → `CloudApiDefaults.parse`) builds a BARE `GoApi(host, port)` (key=null, platform=null, storage=null) → id `GoApi(host:port)`;
- the on-device / mDNS-discovered flow (`OnboardingViewModel.kt:710 onOnDeviceApiReturned` carries a per-instance `key`; `OnboardingViewModel.kt:239 startApiDiscovery`

carries `platform`+ `storage` TXT attributes) builds a KEYED `GoApi` → id `GoApi(host:port#k=…)`.

Both call `endpointsRepository.add()` → `EndpointDao.insert(OnConflictStrategy.REPLACE)`. REPLACE de-dups ONLY on a matching primary-key id, so the two different ids produce TWO Room rows for the one server. `EndpointsRepositoryImpl.observeAll()` mapped the DAO list straight through with no de-dup by server identity, so the Connections/ Server list (`feature/connection/.../ConnectionsViewModel.kt:79` `observeConnections` → `ObserveEndpointsStatusUseCase`) rendered the same host:port twice.

Affected files: `core/data/src/main/kotlin/lava/data/impl/repository/EndpointsRepositoryImpl.kt` (fix), `core/data/src/test/.../EndpointsRepositoryImplFilterTest.kt` (tests).

Fix: `EndpointsRepositoryImpl.observeAll()` now applies `.distinctBy(::serverIdentity)` after the model-map / Rutracker-filter step. `serverIdentity(GoApi)` is `"GoApi(host:port)"` — the transport-defining identity, deliberately EXCLUDING the additive `key`/`platform`/`storage` fields that bloat the Room id — so one row shows per actual server regardless of which path added it. `distinctBy` keeps the first occurrence (Room emits in insertion order). Per-endpoint auth is unaffected: the active endpoint's own key is persisted separately by `lava.securestorage.model.EndpointConverter`, not by this list path.

Verification test/challenge: `EndpointsRepositoryImplFilterTest.observeAll_deduplicates_same_server_added_via_two_paths` (new) — inserts the same host:port via a bare GoApi and a keyed GoApi, asserts the emitted list contains that server exactly once. Companion `observeAll_keeps_distinct_servers` guards that two genuinely different servers both survive (the de-dup is by identity, not a blanket collapse). Bluff-Audit: with the `.distinctBy(::serverIdentity)` line removed, the test FAILED `java.lang.AssertionError: The chosen online server MUST appear exactly ONCE in the Server list, not twice (operator defect 2026-06-14) expected:<1> but was:<2>` (`EndpointsRepositoryImplFilterTest.kt:125`); restored → BUILD SUCCESSFUL (5/5).

Fix commit: this commit. **Forensic anchor:** operator-reported 2026-06-14; reproduced before fix per §6.T.1. Version files NOT touched.

search "Something went wrong" across providers via the on-device/LAN API (2026-06-14)

Symptom (operator-reported): onboarded RuTracker + YTS + Kinozal, searched "prince" on Home with all 3 selected, using the Android (on-device) API as the endpoint → "Error: Something went wrong, please try again." Every provider failed.

Root cause (CONFIRMED — same class as SP-3.1 / Defect-A, fixed for /providers but never for per-provider ops): `ApiBackedTrackerClient` per-provider requests (search/getString/getBytes/postJson/healthCheck) were built on the UNQUALIFIED, strict, system-trust `OkHttpClient` and carried NO per-instance auth key. The on-device / LAN API is self-signed (so the system-trust client fails the TLS handshake) AND auth-gated by the `Lava-Auth` header (so even on plain HTTP it 401s without the per-instance key). Either way the SDK surfaced the generic "Something went wrong." `NetworkModule.kt:101` documents that the unqualified client "MUST NOT be used for LAN endpoints"; only the `@Named("lan")` permissive client may. `/providers` had been migrated to the LAN client + key, but per-provider ops had not.

Affected files: `core/tracker/client/.../ApiBackedTrackerClient.kt` (+ `withAuth()` on every request), `.../ApiBaseUrlHolder.kt` (per-endpoint key), `.../di/TrackerClientModule.kt` (factory now injects `@Named("lan")` client + `authFieldName` + `ApiBaseUrlHolder.currentKey()`), `feature/onboarding/.../OnboardingViewModel.kt` (`ApiBaseUrlHolder.set(url,key)`), `core/domain/.../RepopulateProvidersOnStartupUseCase.kt` + `app/.../StartupProvidersModule.kt` (cold-start key thread-through).

Fix: the active-API base URL is now stored WITH its per-instance key in `ApiBaseUrlHolder`; `TrackerClientModule` builds every `ApiBackedTrackerClient` with the `@Named("lan")` permissive-TLS client and threads `authFieldName`+`authKey`; `withAuth()` attaches `Lava-Auth: <key>` to every per-provider request. So search/browse/download against a self-signed, auth-gated on-device API succeed instead of failing the handshake or 401ing.

Verification test/challenge:

- Unit (reproduction): `ApiBackedTrackerClientTest.search_attachesPerEndpointAuthKey_soAuthGatedApiReturnsResults` (on-device MockWebServer 401s without `Lava-Auth: k`, returns results with it)
  - `search_withoutAuthKey_throwsOnAuthGatedApi`. Bluff-Audit: dropping `.withAuth()` → search 401s → test FAILED; restored → GREEN.
- Wiring contract: `LanHttpClientWiringContractTest` reflectively asserts the factory's OkHttpClient param is `@Named("lan")`

(closes the gap that nothing asserted the registry was wired with the permissive client).
- Device: Challenge44ApiSearchAuthTest (on-device MockWebServer) — PASS on the Genymotion VM in the 1.3.8-1065 client gate (BUILD SUCCESSFUL).

Fix commit: landed in the 1.3.8-1065 cycle (search-auth thread-through). **Forensic anchor:** operator-reported 2026-06-14; reproduced before fix (§6.T.1).

onboarding "Pick your providers": select-all / deselect-all control (2026-06-14)

Symptom (operator request): "On pick your providers onboarding screen we must have check and uncheck all checker, especially if there are multiple choices in dozens." No way to toggle every provider at once.

Fix: added `OnboardingAction.ToggleAllProviders` + `OnboardingViewModel.onToggleAllProviders()` (computes a single target = `!(all currently selected)` and maps every provider to it — so it acts as select-all when not all are selected, deselect-all when all are). `ProvidersStep` renders a `select_all_providers`-tagged control when `providers.size >= 2`. **Affected files:** `feature/onboarding/.../OnboardingAction.kt`, `OnboardingViewModel.kt`, `steps/ProvidersStep.kt`, `OnboardingScreen.kt`. **Verification:** Challenge41OnboardingSelectAllProvidersTest — PASS on the Genymotion VM (1.3.8-1065 gate). Drives the real ProvidersStep, taps select-all, asserts all selected; taps again, asserts all cleared. **Fix commit:** 1.3.8-1065 cycle.

onboarding provider Configure: password field masking + eye toggle (2026-06-14)

Symptom (operator request): "password field MUST behave like every regular password field — masking letters, with eye icon to show password." The Configure step's password was plain text.

Fix: `ConfigureStep` password field now uses `visualTransformation = if (passwordVisible) None else PasswordVisualTransformation()`

- `KeyboardType.Password` + a trailing eye control (`password_visibility_toggle`, content descriptions "Show password"/"Hide password") that flips `passwordVisible`. **Affected files:** `feature/onboarding/.../steps/ConfigureStep.kt`.

Verification: Challenge42OnboardingPasswordMaskingTest — PASS on the Genymotion VM (1.3.8-1065 gate). NOTE (anti-bluff): Compose retains the RAW text in EditableText regardless of the visual mask, so `assertDoesNotExist(plaintext)` is INVALID — the test asserts the masking STATE via the eye control's content-description round-trip (Show→Hide→Show). The original `assertDoesNotExist` form FAILED on device; the device run caught the test bug (a real §6.Z win) and it was rewritten to the content-description contract. **Fix commit:** 1.3.8-1065 cycle (test corrected in c366454f).

api-app foreground service crashes after ~6h — dataSync → specialUse (2026-06-14)

Symptom (Crashlytics FATAL): `ForegroundServiceStartNotAllowedException` + `ForegroundServiceDidNotStopInTimeException` on the long-lived on-device API server after ~6h uptime (Galaxy S23 Ultra / Android 16). Issues `9ba8502ee0ba0d1fdd03987650b8acf8` + `b9baeaede585fc3bc9b515c27cde532c`.

Root cause (CONFIRMED): `ApiEngineService` used `foregroundServiceType="dataSync"`, which Android 14+ caps at ~6h cumulative runtime/24h; the long-lived LAN API server exhausts the budget and `startForeground` then throws.

Fix (operator-approved specialUse): `foregroundServiceType` `dataSync` → `specialUse` (no time budget) + `FOREGROUND_SERVICE_SPECIAL_USE` permission + `PROPERTY_SPECIAL_USE_FGS_SUBTYPE` (Play-review justification). Defensive: `startForeground` wrapped in `try/catch (ForegroundServiceStartNotAllowedException)` → graceful `stopSelf`, and an `onTimeout(startId, fgsType)` override stops cleanly — so even a future budget degrades gracefully instead of crashing. **Affected files:** `api-app/src/main/AndroidManifest.xml`, `api-app/.../service/ApiEngineService.kt`. Design: `docs/issues/2026-06-14-apiapp-fgs-datasync-budget-fix-design.md`. **Verification:** `:api-app:compileDebugKotlin` SUCCESSFUL + the 0.2.8-12 release FGS `specialUse` cold-start canary on the Genymotion VM (the service starts under specialUse with no FGS exception — a misconfigured type would throw at startForeground → FATAL). §6.O closure log: `.lava-ci-evidence/crashlytics-resolved/2026-06-14-apiapp-fgs-datasync-budget.md`. **Fix commit:** `ed03cac2` (cherry-picked to master from 10f39f43).

Defect B — api-app Crashlytics telemetry misattributed to the CLIENT Firebase app (2026-06-14)

Symptom: api-app crashes/non-fatals (incl. the FGS FATALs above) landed under the CLIENT Firebase app's dashboard, not the api-app's own app — so api-app telemetry was indistinguishable from client telemetry.

Root cause (CONFIRMED via Firebase MCP): the tracked (gitignored) `api-app/google-services.json` mapped the api-app package names (`digital.vasic.lava.api` / `.api.dev`) to the CLIENT app's `mobilesdk_app_id` values. The `google-services` Gradle plugin matches the `client[]` entry by `package_name`, so the api-app build baked in the client app id and registered as the client app.

Fix: replaced `api-app/google-services.json` (gitignored, on-disk only — never committed per §6.H) with the correct config mapping the api-app packages to the real "Lava API (release)" `...d57b960e955645f6cfd20a` / "Lava API (debug)" `...2932451e07ca80a7cfd20a` app ids, fetched via Firebase MCP from project `lava-vasic-digital` (both api-app Firebase apps already exist — no creation needed). api-app rebuilt + re-staged for 0.2.8-12. **Affected files:** `api-app/google-services.json` (gitignored secret — NOT committed). Findings: `docs/issues/2026-06-14-defect-b-telemetry-attribution-findings.md`. **Verification:** api-app rebuilt SUCCESSFUL with the corrected config + the FGS cold-start canary passes on the rebuilt artifact. Once 0.2.8-12 is distributed + observed, api-app crashes will appear under the api-app dashboard (operator confirms post-distribute). **Fix commit:** this cycle (config replaced on disk; findings doc `fb0bfec9`).

search does not work in any scenario — 5-layer cascade (2026-06-14)

> **STATUS (honest, §6.T.1/§11.4.6): search does NOT yet work on-device.** Layers 1+2+3 are fixed (L1+L2 device-verified, L3 RED→GREEN at the test layer). Layer 4 is proven a TEST-VM artifact (production grant is safe — clean matched-pair run showed `granted=true`). **Layer 5 is OPEN/VERIFYING** — even on a clean matched pair with the permission granted and the engine running, `/v1/{provider}/search` STILL 401s; the `Lava-Auth` key is not reaching the search request. A logcat-instrumentation run is in progress. **1.3.9 (client-only) is NOT shipped** until the L5 on-device gate is GREEN. api-app unchanged at 0.2.8-12. This entry

> was extended from 3 to 5 layers as the cascade was peeled back; do not read any "fixed" below as "search works" — it does not yet.

Symptom (operator-reported, 1.3.8-1065): "search still does not work in any scenario." Searching any query against the chosen on-device / LAN API returned no results across every provider. The HelixQA video-QA agent could not produce a search-results walkthrough because search genuinely failed.

Root cause (CONFIRMED — a CASCADE of three independent bugs, all proven from on-device evidence, not one):

- **Layer 1 — onboarding never persisted the chosen API to `settings.endpoint`.** `OnboardingViewModel.onSelectApi` wrote the selected API to the Room `Endpoint` list + `ApiBaseUrlHolder` (what `GET /providers` + the dynamic SDK clients read) but NEVER to `settings.endpoint`. The home search resolves its target host from `SettingsRepository.getSettings().endpoint` (`NetworkApiRepositoryImpl.endpoint()`), which therefore stayed at the default `Endpoint.GoApi("lava-api.local")` → `UnknownHostException` (the host never resolves on the LAN). So search targeted the wrong host even though `/providers` worked.
- **Layer 2 — GoApi search routed to `GET /v1/search`, a route NO backend serves.** `SearchResultViewModel`'s dispatch sent every GoApi multi-provider search to `observeSseSearch` → `GET {base}/v1/search`. Verified across the whole Go codebase: `internal/router/router.go` serves `/providers` + `/v1/:provider/{op}` + `/jackett/search` + `/health/ready`; the standalone serves legacy `/search` + `/v1/{provider}/search`; **neither registers `/v1/search`** → 404 for every GoApi user.
- **Layer 3 — mDNS-discovered endpoint persisted without its `Lava-Auth` key (FIXED, `21031f2a`).** `onSelectApi` adopted a discovered API's host/port but never attached the locally-readable key, so the persisted `settings.endpoint` was keyless → `/v1` ops 401 even when the key was readable. Fix: `onSelectApi` reads the local api-app key for keyless mDNS endpoints + an app-side cold-start key-restore heals an already-persisted keyless endpoint. Both RED→GREEN at the test layer.
- **Layer 4 — `READ_API_KEY` signature permission not granted on the test VM (TEST-VM ARTIFACT, production-safe).** After L3, the key reader still returned null because the signature permission `digital.vasic.lava.permission.READ_API_KEY` was not granted to the client on the VM. Root cause (proven, full evidence in `docs/issues/2026-06-14-readapikey-permission-variant-analysis.md`): the permission name is a **fixed literal** (no `.dev` suffix) while every other cross-app identifier is variant-suffixed, and the VM had BOTH differently-signed api-app variants (debug `…api.dev` + release `…api`) in its install history → the Android

`INSTALL_FAILED_DUPLICATE_PERMISSION` trap denies the grant. **Production is SAFE** — the shipped release-client + release-api-app pair shares one signing cert and grants the permission at install; **a clean matched-pair install confirmed `granted=true` on-device.** Real users never co-install debug
    - release variants. A MEDIUM-priority QA-fidelity hardening (variant-suffix the permission name, like the authority already is) is recommended so the test VM exercises the production grant path — not shipping-blocking.
- **Layer 5 — `Lava-Auth` key not reaching the `ApiBackedTrackerClient` search request (OPEN / VERIFYING — the load-bearing open layer).** On a CLEAN matched pair (release client + release api-app, same signature, `granted=true`, engine running, fresh onboard), `/v1/{provider}/search` STILL returns 401. The key is granted and readable but is not reaching the search request. **Evidence caveat:** a public `GET /providers` → 200 proves reachability ONLY — `/providers` is registered BEFORE the auth middleware, so a 200 there does NOT prove the key is present/valid; only an authed `/v1/{provider}/…` exercises the key. A logcat-instrumentation run is in progress to pinpoint exactly which link in the key-flow drops the key. **Until this closes, search does NOT work on-device** and no layer above it makes search functional on its own.

Affected files:

- `feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingViewModel.kt` (`onSelectApi` now calls `SetEndpointUseCase` to persist the active `settings.endpoint`; nullable injected seam, real impl bound by Hilt).
- `core/domain/src/main/kotlin/lava/domain/usecase/RepopulateProvidersOnStartupUseCase.kt` (`reconcileActiveEndpoint()` heals existing installs whose onboarding pre-dated the Layer-1 fix — adopts the Room GoApi when `settings.endpoint` is an orphan default).
- `feature/search_result/src/main/kotlin/lava/search/result/SearchResultViewModel.kt` (GoApi multi-provider search re-routed from `observeSseSearch` → `/v1/search` to `observeStreamMultiSearch` → `sdk.streamMultiSearch` → `GET /v1/{provider}/search`, the served path that carries the per-instance `Lava-Auth` key + permissive-LAN client from the 1.3.8 wiring). The dead `observeSseSearch` → `/v1/search` consumer + its 3 SSE bluff tests were deleted in `addaacd0`.
- `feature/onboarding/.../OnboardingViewModel.kt` + `MainActivity` key-restore (Layer 3, `21031f2a`): `onSelectApi` reads the local api-app key for keyless mDNS-discovered endpoints; cold-start key-restore heals a persisted keyless endpoint.

Verification tests added (real-stack reproduction, falsifiability-rehearsed):

- `OnboardingViewModelDynamicProvidersTest` — `selecting an API persists it as the active settings endpoint the search path reads` (real ViewModel + `SetEndpointUseCase` seam; reproduces Layer 1: without the persist write, the active endpoint stays the orphan default).
- `RepopulateProvidersOnStartupUseCaseTest` — `cold start heals a stale orphan active endpoint to the onboarded server in the list` (real use case; reproduces the existing-install heal path).

On-device Chucker evidence (1.3.8 debug client, Genymotion VM): `GET https://10.0.3.16/providers` → 200; `GET https://lava-api.local/search?query=prince…` → `UnknownHostException` (host never resolves; api-app "Requests served: 0"). Traced via the flow-db `Endpoint` row + `chucker.db` transactions; surfaced by the HelixQA video-QA agent.

§6.J bluff removed: `SearchResultSseErrorRetryTest` drove `observeSseSearch` over a `MockWebServer` that SERVED `/v1/search` — the route no backend registers — and asserted SSE error/retry rendering. It passed green while the real feature 404'd (Seventh Law clause 4: mock served the endpoint production 404s). Removed. Incident: `.lava-ci-evidence/sixth-law-incidents/2026-06-14-sse-search-v1search-unserved-bluff.json`.

Fix commits: `d05bc71e` (L1+L2), `21031f2a` (L3 key restore), `addaacd0` (L4 cleanup — dropped ignored authority param + dead SSE path).

§6.J meta-lesson (this cycle): a verification subagent proposed an L4 fix — a hardcoded `".keyprovider"` authority string — and the main stream REJECTED it after reading the source: `MainActivity.buildApiKeyReader` returns `{ -> client.read()?.key }`, which takes no authority arg and ignores any passed. The proposed change would not have touched the real grant condition (the signature match) → a §6.J bluff (and a §6.R hardcoded literal). Verify subagents root-cause; the main stream verifies a proposed change reaches the failing path before applying.

REMAINING / OPEN / VERIFYING (honest, per §6.T.1 / §11.4.6 — search does NOT yet work on-device):

- **Layer 5 (OPEN, top priority):** clean matched pair, `granted=true`, engine up, fresh onboard → `/v1/{provider}/search` STILL 401; the `Lava-Auth` key is read but not delivered on the search request. Logcat-instrumentation run in progress to find the drop point. **1.3.9 (client-only) NOT shipped until this gate is GREEN.** api-app unchanged at 0.2.8-12.
- **Layer 4 (OWED, MEDIUM/QA-fidelity):** variant-suffix the `READ_API_KEY` permission name so the test VM exercises the

production grant path; production grant is already safe. Not shipping-blocking.
- Existing installs that do NOT re-onboard get a keyless healed endpoint → `/v1` ops 401 (the per-instance key lives only in `settings.endpoint` via `EndpointConverter`, never in the Room `Endpoint` row). Fresh onboard flows the key correctly; the L3 cold-start key-restore addresses keyless mDNS endpoints, but a full existing-install key-restore is still **owed**.
- Dead SSE path **CLOSED** (`addaacd0`): `observeSseSearch` + the 3 SSE bluff tests serving the unserved `/v1/search` were removed. A server-side `/v1/search` SSE aggregator, if ever wanted, is a fresh feature, not a revival.

auth-provider search — Auth-Token (login session) not threaded onto dynamic /v1 requests (2026-06-14, P0-1)

Symptom: after the 5-layer search fix shipped (1.3.9), search returned real results for NO-AUTH providers (Internet Archive) but NOT for login-required RuTracker/Kinozal (the operator's original report). **Root cause:** the Go `/v1/{provider}/search` path needs TWO credentials — `Lava-Auth` (per-instance key, fixed in 1.3.9) AND `Auth-Token` (the provider login session, `{provider}:cookie:{session}`, parsed in `lava-api-go/internal/handlers/handlers.go:118`). `ApiBackedTrackerClient.withAuth()` attached only `Lava-Auth`, so authed providers reached the server anonymous (`Type:"none"`) → empty/login. **Fix:** new `ProviderSessionTokenHolder` seam (parallel to `ApiBaseUrlHolder`); `ProviderLoginViewModel` writes the session on login; `TrackerClientModule`'s factory reads it; `ApiBackedTrackerClient.withAuth()` attaches `Auth-Token: {trackerId}:cookie:{session}` IN ADDITION to `Lava-Auth` when present. Additive — no-auth providers attach nothing, unchanged. **Affected files:** `core/tracker/client/.../ApiBackedTrackerClient.kt`, `.../ProviderSessionTokenHolder.kt` (new), `.../di/TrackerClientModule.kt`, `feature/login/.../ProviderLoginViewModel.kt`. **Verification:** `ProviderSessionTokenEndToEndWiringTest` — real holder→factory→`withAuth` over MockWebServer with an auth-gated dispatcher; PRIMARY assertion `RecordedRequest.getHeader("Auth-Token")` present for authed / absent for no-auth. Bluff-Audit: drop the attach → `authProviderSession_isThreaded_ontoTheAuthTokenHeader` FAILED (Auth-Token absent → 401); reverted → :core:tracker:client + :feature:login GREEN. **ON-DEVICE VERIFY WITH REAL RUTRACKER CREDS OWED** (subagent rate-limit) — shipped in 1.3.10 for tester verification. **Fix commit:** `3b1b6a14`.

remote/cloud GoApi wrongly given the LOCAL api-app key (2026-06-14, P1-4)

Symptom (edge): `OnboardingViewModel.withLocalApiKeyIfMissing()` read the LOCAL on-device api-app key for ANY keyless `Endpoint.GoApi` — correct for the local api-app, but a remote/cloud GoApi would get the local key → 401. No tests covered it. **Fix:** gate the key-read on `String.isLocalHost()` (loopback / RFC-1918 / `.local`); remote hosts stay keyless. **Affected:** `feature/onboarding/.../OnboardingViewModel.kt` + test. Bluff-Audit: remove the `isLocalHost` guard → remote endpoint wrongly keyed → test FAILED; reverted → GREEN (6/6). On-device mDNS flow not regressed. **Fix commit:** `b3cb6de2`.

READ_API_KEY permission not variant-suffixed → debug+release collide (2026-06-14, P2-1)

Symptom (QA-fidelity/security): the signature permission `digital.vasic.lava.permission.READ_API_KEY` was a fixed literal (the authority was already variant-suffixed). A device with BOTH debug+release variants co-installed hit `INSTALL_FAILED_DUPLICATE_PERMISSION` / broken grant — which cost hours of test-VM debugging in the search cycle (it masked the real grant path). **Fix:** `${apiKeyPermission}` manifest placeholder per build type — release byte-identical `...permission.READ_API_KEY` (existing grants survive), debug `...permission.dev.READ_API_KEY`. Both manifests merge-verified; `ApiKeyProvider.attachInfoForTest` now reads the variant-aware `BuildConfig.API_KEY_PERMISSION`. **Affected:** `app/build.gradle.kts`, `api-app/build.gradle.kts`, both `AndroidManifest.xml`, `ApiKeyProvider.kt`. **Fix commit:** `4026756c`. Analysis: `docs/issues/2026-06-14-readapikey-permission-variant-analysis.md`.

2026-06-23 — H1 search-401: AuthInterceptor overwrote per-install handoff key

Root cause: `AuthInterceptor` in `core/network/impl/` attached the build-time `LAVA_AUTH` UUID using OkHttp's `.header(fieldName, value)` (replace semantics). Interceptors fire last in the OkHttp chain, so it unconditionally overwrote the per-install handoff key that `ApiKeyClient` had already placed on the request under the

same `Lava-Auth` header name. The engine received the wrong (build-time) credential and returned HTTP 401, which the app surfaced as "Something went wrong" on every search.

Affected files: `core/network/impl/src/main/kotlin/lava/network/impl/AuthInterceptor.kt`

Fix: Added an only-if-absent guard — the interceptor attaches the build-time UUID only when the request does not already carry the `Lava-Auth` header, so the per-install handoff key survives to the wire unchanged.

Verification test/challenge: `core/network/impl/.../AuthInterceptorHandoffKeyTest.kt` (JVM, 5 tests, falsifiability-proven: reverting the guard caused `whenPreExistingHandoffKeyIsPresent_interceptorMustNotOverwrite` to fail with `expected:<[HANDOFF-KEY-B64]> but was:<[<build-time-UUID-B64>]>`). Challenge45 (`C45`) exercises the full on-device search-auth flow end-to-end.

Fix commit: `b58ef78b`

Forensic anchor: `docs/issues/2026-06-23-search-401-rootcause-deepening.md`; incident surfaced during operator testing of every search returning "Something went wrong".

2026-06-23 — api-app-17-release stale-binary (§6.Z wrong-binary)

Root cause: The 1071 rebuild failed mid-package at `:app:uploadCrashlyticsMappingFileRelease` (transient Crashlytics DNS error) before `:api-app:assembleRelease` finished. `:api-app:clean` was not run before the next attempt, so the leftover output file was named `*-17-*` (matching the new version name) but its embedded `android:versionCode` still declared `16` from the previous build. The `firebase-distribute.sh` filename-only picker matched the filename and distributed the stale binary; the api-app release channel shipped versionCode 16 as release 17.

Affected files: `scripts/firebase-distribute.sh`

Fix: (1) api-app rebuilt clean (`./gradlew :api-app:clean :api-app:assembleRelease`) and distributed as version 18 (corrective). (2) `firebase-distribute.sh` hardened with a `_assert_apk_versioncode` guard that `aapt2`-dumps the picked APK's manifest and fatals if its actual `versionCode` differs from `APP_VERSION_CODE` — the filename matches the name, this gate matches the bytes.

Verification test/challenge: `tests/firebase/test_assert_apk_versioncode.sh` (hermetic; exercises the new aapt content-guard with a positive case and a stale-versionCode negative case).

Fix commit: `b58ef78b`

Forensic anchor: `.lava-ci-evidence/sixth-law-incidents/2026-06-23-apiapp-17-release-stale-binary.json`; discovered by the conductor's own post-distribute aapt sweep, not a user report.

2026-06-23 — ApiHttpException type regression (IOException vs IllegalStateException)

Root cause: The §6.AC telemetry commit (`68b6e650`) introduced `ApiHttpException` as a typed HTTP-error wrapper extending `IOException`. Four existing tests in `core:tracker:client` asserted the historical contract that HTTP errors from `ApiBackedTrackerClient` extend `IllegalStateException`. The `IOException` supertype broke those assertions, causing 4 test failures surfaced during the integration test run after the telemetry commit landed.

Affected files: `core/tracker/client/src/main/kotlin/lava/tracker/client/ApiBackedTrackerClient.kt`

Fix: Changed `ApiHttpException` to extend `IllegalStateException` instead of `IOException`, preserving the original HTTP-error contract while retaining the structured `statusCode`/`requestUrl`/`httpMethod` context fields needed for §6.AC telemetry enrichment.

Verification test/challenge: The 4 pre-existing `core:tracker:client` tests asserting the `IllegalStateException` HTTP-error contract; all 4 GREEN in the 860/0 full JVM suite attested at commit `a6d8cbf7`. Regression surfaced and fixed within the same session (caught by integration testing before distribute).

Fix commit: `68b6e650`

Forensic anchor: Caught during the full `./gradlew testDebugUnitTest` run after the §6.AC telemetry commit; not a user report.

2026-06-23 — ApiKeyClientTest compile break (§6.AC analytics-param addition)

Root cause: The §6.AC telemetry commit (`68b6e650`) added an `analytics: AnalyticsTracker` constructor parameter to `ApiKeyClient`. `ApiKeyClientTest` was not updated in the same commit, so it stopped compiling. The break was not caught by the session's targeted test runs (which built `androidTest` APKs, not `:app` JVM unit tests) and was only surfaced by the subsequent full-suite attestation run.

Affected files: `app/src/test/kotlin/lava/vasic/lava/client/handoff/ApiKeyClientTest.kt`

Fix: Passed a no-op `AnalyticsTracker` boundary stub to `ApiKeyClient`'s constructor in the test. Telemetry is an outermost boundary per §6.AC; no SUT was mocked and no production code was touched. The test's 3 existing key-read assertions were byte-unchanged and remain GREEN.

Verification test/challenge: `ApiKeyClientTest` (3 tests); GREEN in the 860/0 full Android JVM suite attestation at commit `a6d8cbf7` (`.lava-ci-evidence/test-runs/2026-06-23-android-jvm-suite.md`).

Fix commit: `a6d8cbf7`

Forensic anchor: Surfaced by the comprehensive full-suite attestation run that followed targeted C44/C45 on-device validation; not a user report.

2026-06-23 — firebase-distribute SIGPIPE on head-7 truncation (process discipline)

Root cause: A `head -7` truncation inserted in the distribute loop to limit output caused `SIGPIPE` to kill the upstream distribute process before it could write the `last-version-debug` pointer file. The first client-debug distribute of version 1072 appeared to succeed visually (Firebase upload completed) but left the pointer unwritten; the subsequent `--release-only` stage saw a stale pointer and blocked on the §6.AA release-after-debug-confirm gate.

Affected files: `scripts/firebase-distribute.sh` (process discipline note; the `head -7` was removed from the distribute loop output path).

Fix: Removed the `head -7` truncation from the distribute pipeline so the process completes cleanly before the shell closes the write end of the pipe. Pointer files are now written atomically after the full distribute subprocess exits.

Verification test/challenge: The §6.Z aapt content-guard (`_assert_apk_versioncode`) confirmed all 4 binaries of the 1072 cycle distributed correctly after the fix (`.lava-ci-evidence/.../2026-06-23-C00-1072-PASS.json`). No dedicated hermetic test added (the SIGPIPE is a shell process-discipline issue, not a logic branch in the script).

Fix commit: `5dd7ae06`

Forensic anchor: Observed in distribute loop output during the 1072 all-4 distribute cycle; re-run of client-debug distribute recorded correct `last-version-debug 1072` pointer.

BUG-2026-06-24-A — Search streaming: back-press hang + no client-side timeout

Date: 2026-06-24 **Surface:** `feature/search_result` — `SearchResultViewModel` **Crashlytics §6.O closure log:** `.lava-ci-evidence/crashlytics-resolved/2026-06-24-search-timeout-cancel.md`

Root cause (Bug 1 — no cancellation on back-press): `observeStreamMultiSearch()` starts an Orbit `intent {}` coroutine that blocks on `sdk.streamMultiSearch(...).collect {}`. Back-press dispatches a *new* `intent {}` (`onBackClick`) that only posts a `Back` side effect — it does NOT cancel the in-flight coroutine. On a slow provider (e.g. yts), the user taps Back, the screen navigates, but the streaming coroutine stays alive until OkHttp's 30 s `readTimeout` fires. User sees a ~30 s hang during which Back appears unresponsive.

Root cause (Bug 2 — no client-side timeout): `observeStreamMultiSearch()` had no `withTimeout` guard. A slow-but-not-stalled provider holds the `SearchResultContent.Streaming` spinner for up to 30 s with no Retry affordance — the `handleStreamEnd()` → `Error` path only triggers once `collect {}` returns.

Root cause (Bug 3 — engine unbounded search handler + stale YTS mirrors): `lava-api-go/internal/handlers/v1/search.go` passed `c.Request.Context()` (no deadline) to `p.Search()`. YTS failover loop: 4 mirrors × `perAttemptTimeout(8 s)` = up to 32 s, exceeding OkHttp's 30 s `readTimeout` on the client — direct source of the `SocketTimeoutException` in Crashlytics issue `9d4ad2f4…`. Independently, the YTS mirror list led with `yts.mx` which is NXDOMAIN (confirmed 2026-06-24), forcing the client to burn all failover slots before timing out. Field evidence: Crashlytics key

provider=yts, exception at Http2Stream.takeHeaders (read-headers hang, not a write/connect failure). Investigation doc: docs/issues/2026-06-24-search-timeout-and-interrupt-rootcause.md.

Affected files:

- feature/search_result/src/main/kotlin/lava/search/result/SearchResultViewModel.kt (Bug 1 + Bug 2 — cancel-on-back + withTimeout)
- lava-api-go/internal/handlers/v1/search.go (Bug 3 — context.WithTimeout(18 s) deadline wrapping p.Search() at line 69)
- lava-api-go/internal/provider/curated/yts/client.go (Bug 3 — DefaultBaseURLs refreshed: dead yts.mx dropped, 5 live mirrors: yts.bz, yts.lt, yts.am, yts.gg, movies-api.accel.li)

Fix:

- **Bug 1 (cancel-on-back):** Added @Volatile private var activeSearchJob: Job? = null. Inside observeStreamMultiSearch() captured the coroutine Job via currentCoroutineContext()[Job] and stored in activeSearchJob. onBackClick() now cancels any prior activeSearchJob before posting Back.
- **Bug 2 (client timeout):** Wrapped collect {} in withTimeout(SEARCH_TIMEOUT_MS) (25 000 ms, 5 s below OkHttp's readTimeout); TimeoutCancellationException caught locally, marks still-SEARCHING providers as StreamStatus.ERROR, routes to SearchResultContent.Error with Retry button. Generic Exception catch with rethrowIfCancellation() guard handles connectivity-lost similarly. §6.AC telemetry in both catch blocks.
- **Bug 3 (engine deadline + mirror refresh):** handlers/v1/search.go:69 wraps p.Search() in context.WithTimeout(c.Request.Context(), 18*time.Second) so the engine ALWAYS returns (results or fast error) before the client's 30 s deadline. yts/client.go DefaultBaseURLs refreshed to 5 live mirrors (dead yts.mx dropped). Defensive prior-job cancel added in observeStreamMultiSearch() for future double-invocation safety (cleanup 1aa42536).

Verification tests:

Client (Android): feature/search_result/src/test/kotlin/lava/search/result/SearchResultViewModelCancelTimeoutTest.kt — 3 tests using REAL SearchResultViewModel + REAL LavaTrackerSdk + REAL DefaultTrackerRegistry; only the outermost network boundary faked via CompletableDeferred-based clients (dispatcher-agnostic indefinite suspension):

1. back_press_while_streaming_cancels_inflight_search_and_emits_Back
2. slow_provider_exceeding_timeout_surfaces_Error_with_Retry_affordance

3. `back_press_after_completed_search_preserves_Content_and_emits_Back`

Engine (Go):

- `lava-api-go/internal/provider/curated/yts/search_total_deadline_test.go` — `TestSearch_ExceedsDeadline_ReturnsContextDeadlineExceeded` (slow mock server + 18 s ctx deadline → confirms the engine surfaces an error, not a hang) + `TestSearch_WithinDeadline_ReturnsResults`.
- `lava-api-go/internal/provider/curated/yts/search_total_deadline_test.go` — `TestReproduction_StaleMirrorListFails` (NewClientWithMirrors(["yts.mx"]).Search → provider error; fresh list → real results in <2 s).

Falsifiability confirmed (§6.N / Seventh Law clause 1):

- Removed `activeSearchJob?.cancel()` → Test 1 `isStillCollecting` stays `true` → FAIL
- Removed `withTimeout(SEARCH_TIMEOUT_MS)` → Test 2 times out waiting for `Error` → FAIL
- Reverted `DefaultBaseURLs` to `yts.mx`-only → `TestReproduction_StaleMirrorListFails` reproduces `yts.mx: provider: unknown error` → FAIL; fresh list → PASS (Bluff-Audit in commit body `0e81730b`).

Fix commits:

- `20d98914` — client Bug 1 (cancel-on-back) + Bug 2 (25 s `withTimeout`) + engine Bug 3 (18 s handler deadline in `handlers/v1/search.go:69`)
- `0e81730b` — YTS stale mirror refresh (`DefaultBaseURLs` + `TestReproduction_StaleMirrorListFails`)
- `1aa42536` — code-review cleanups (defensive prior-job cancel; mirror-count comment fix)
- `3c1fa159` — version bumps: lava-api-go 2.3.33, CHANGELOG / snapshots for 1.3.11-1073 / api-app 0.2.11-19

Forensic anchor: Firebase Crashlytics NON_FATAL `9d4ad2f4d1a8b8697b1506402e045b81` (client release app `1:815513478335:android:456475e2ef4039d8cfd20a`), `SocketTimeoutException` at `Http2Stream.takeHeaders`, keys `feature=search / operation=streamMultiSearch / provider=yts`, device HUAWEI TXZ-W09 / Android 12. Operator report (2026-06-24): "search does not work — no results; can't go back or interrupt it." Coordination analysis: `docs/issues/2026-06-24-search-timeout-coordination-analysis.md`. §6.O closure log: `.lava-ci-evidence/crashlytics-resolved/2026-06-24-search-socket-timeout.md`

Challenge Test: owed — `C-SEARCH-CANCEL` per §6.AE (tracked in `docs/workable_items.db`). Parallel-authored on-device Challenge is the §6.O clause-2 e2e gate.

2026-06-24 — P0 CredentialsKeyHolder locked FATAL (Crashlytics 58a1335272bc)

Root cause: `CredentialsEntryRepositoryImpl.observe()` called `keyProvider()` (which invokes `CredentialsKeyHolder.require()`) unconditionally inside a Room `Flow.map` operator. When the key holder is in the **locked** state (normal at fresh app start or after session expiry), `require()` throws `IllegalStateException("credentials key holder is locked …")`. That exception escaped the `map` operator and propagated onto the **main-thread Looper** via the ViewModel's `combine` collector, causing a FATAL crash. The user was actively submitting searches (`lava_search_submit: mumy`, then `lava_search_submit: prince`) when the DAO emitted a background update that triggered the observation chain. 5 events / 1 user in 1.3.10.

Affected files:

- `core/credentials/src/main/kotlin/lava/credentials/CredentialsEntryRepositoryImpl.kt` — `observe()` now wraps `keyProvider()` in `runCatching`; locked-ISE → `emptyList()` + §6.AC `recordWarning`; all other throwables re-thrown
- `core/credentials/src/test/kotlin/lava/credentials/CredentialsEntryRepositoryImplTest.kt` — new test `observe emits empty list when key holder is locked — no crash on search path` (line 119)

Fix: `runCatching { keyProvider() }` guards the key-acquisition step. `isLockedKeyHolderError` predicate (type `IllegalStateException` + message prefix `"credentials key holder is locked"`) routes the locked state to `emptyList()`. All other exceptions propagate unchanged. The locked state is expected and normal; crashing on it was the bug.

Verification test/challenge:

- Unit: `CredentialsEntryRepositoryImplTest` — `observe emits empty list when key holder is locked` (falsifiable: revert guard → `IllegalStateException` escapes → test FAILS)
- Challenge: C47 `Challenge47CredentialsLockedSearchTest` — OWED (§6.O clause 2 gate)

Fix commit: uncommitted at log creation; ships in 1.3.11-1073
Forensic anchor: Crashlytics issue `58a1335272bc4ee06595bda6302a670a`, 5 FALTALs on Samsung Galaxy S23 Ultra / Android 16 during search, version 1.3.10-1067. **§6.O closure log:** `.lava-ci-evidence/crashlytics-resolved/2026-06-24-credentials-keyholder-locked.md`

2026-06-24 — P1 LazyColumn nested-scroll 2nd site recurring FATAL (Crashlytics c7c8cccad09f) — FIX OWED

Root cause: A `LazyColumn` (or `LazyVerticalGrid`) is rendered inside a parent `Column`/`Box` with `Modifier.verticalScroll(rememberScrollState())`, giving the lazy layout unbounded vertical height. Compose throws `IllegalStateException: Vertically scrollable component was measured with an infinity maximum height constraints…` at composition time. This is the §6.Q forbidden antipattern. The first site (`TrackerSelectorList`) was fixed in 1.2.3 (`.lava-ci-evidence/crashlytics-resolved/2026-05-05-tracker-settings-nested-scroll.md`), but the Crashlytics issue still recurred through 1.3.10-1067 (2026-06-14), confirming a **second independent site** exists. The breadcrumb (`screen_view{MainActivity}` only) does not identify which sub-screen triggered the 1.3.10 event. A §6.Q structural scan across all composables written/modified since 1.2.5 is required to locate the second site.

Affected files: UNCONFIRMED — to be filled in when §6.Q scan completes and fix is applied.

Fix: OWED. Pattern: replace `LazyColumn` with `Column` for bounded lists, OR apply `Modifier.heightIn(max = …)` to constrain the lazy layout's measurement height.

Verification test/challenge:

- §6.Q structural regression test — OWED (targeting the identified composable)
- Challenge Test driving navigation to the offending screen — OWED (§6.O clause 2 gate)

Fix commit: OWED — ships in 1.3.11-1073 (pending §6.Q scan in this cycle) **Forensic anchor:** Crashlytics issue `c7c8cccad09f…`, FATAL, first seen 1.2.3, last seen 1.3.10 (2026-06-14), Samsung Galaxy S23 Ultra / Android 16. Prior fix for site-1 documented in

`2026-05-05-tracker-settings-nested-scroll.md`. **§6.O closure log:** `.lava-ci-evidence/crashlytics-resolved/2026-06-24-lazycolumn-nested-scroll-2nd-site.md`

2026-06-24 — P2 TopicPageDto MissingFieldException for Internet Archive crawl topics (Crashlytics 8cde0ac208b3)

STATUS: PARTIAL — not fully resolved in 1073 (honest §6.J self-audit correction).

Root cause (corrected after reading the FULL Crashlytics subtitle): the exception lists `Fields [id, title, author, category, torrentData, commentsPage] are required … but they were missing` — kotlinx lists the MISSING fields, so the Internet Archive `WPO-*` crawl-topic `/topic2` response omits **ALL SIX** of `TopicPageDto`'s fields (an archived web page, not a torrent topic). Note `author`/`category`/`torrentData` are nullable but have NO default, so kotlinx still requires the keys present. (An earlier draft of this entry wrongly assumed only `[commentsPage]` was missing — that was based on a reproduction fixture that did not match the real payload.)

Affected files (the partial change that DID land):

- `core/network/api/src/main/kotlin/lava/network/dto/topic/TopicPageDto.kt` — `commentsPage` given default `= TopicPageCommentsDto(page = 1, pages = 1, posts = emptyList())` (line 13)
- `core/network/api/src/test/kotlin/lava/network/dto/topic/TopicPageDtoSerializationTest.kt` — new test file (2 tests, for the commentsPage-only case)

What 1073 does (necessary, NOT sufficient): the `commentsPage` default removes ONE field from the required set. The other five (id, title, author, category, torrentData) remain required, so the real IA crawl-topic payload **still throws** (now listing 5). The 1073 change therefore does not, by itself, stop the reported crash; it is retained as a valid partial improvement.

Why not a rushed blanket fix: defaulting id/title to "" + author/category/torrentData to null would stop the throw but risks masking real malformed-topic bugs for normal rutracker/rutor topics (§6.J). The proper fix (lava-api-go populating the shape for crawl items, OR a dedicated sparse-topic DTO, OR not requesting `/topic2` for IA crawl topics) is a tracked FOLLOW-UP.

Impact + decision: NON_FATAL, 1 event, niche (IA crawl topics only), not a regression — does NOT block 1073 (which fixes the P0 + P1 FATALs + the search timeout). Full fix ships in a later build.

Fix commit (partial): `cb6f76b2` (commentsPage default only); honest correction in the follow-up commit **Forensic anchor:** Crashlytics issue `8cde0ac208b3…`, NON_FATAL, 1 event on Genymobile Pixel 9 / Android 16, version 1.3.9-1066, topic `WPO-20230122202907-crawl897`. **§6.O closure log:** `.lava-ci-evidence/crashlytics-resolved/2026-06-24-topicpagedto-missing-field.md`

LVA-008 — C11/C06 nested-NavHost `search_input` teardown crash (Activity-scoped inner-host lifecycle)

Status: STILL OPEN — **7 app-level candidate fixes now device-FALSIFIED** (2026-06-25). The 6th (Activity-scoped inner-host `LocalLifecycleOwner`, build 1074 @ `1310a922`, evidence `.lava-ci-evidence/1074-gate/`) was reverted and 1075 shipped WITHOUT it. The 7th (Candidate #8 — `launchSingleTop=true` dedupe on `openSearchInput`, branch `lva-008-cand8-gate` @ `e8c81728`) was gated on thinker containerized-KVM: **C06 + C11 reproduced the byte-identical `IllegalStateException`** on the `search/search_input` NavBackStackEntry (destination `0xe36e02dd`) at `MainActivity` destroy (evidence `.lava-ci-evidence/lva008-cand8-gate/`). All 7 app-level candidate classes (nav-version, LenientTeardownRule, nested-host move, atomic popUpTo, NavTeardownGuard, Activity-scoped LifecycleOwner, launchSingleTop dedupe) are exhausted-and-device-falsified — **CONFIRMED upstream androidx-navigation defect** (§11.4.150 research: `b/244910446` family; no fixed-version through nav 2.10.0-alpha04; `docs/research/lva-008-nav-teardown-20260625/`). **8th candidate (Candidate #7 — single-NavHost collapse, branch `lva-008-cand7-singlenavhost` @ `f4e32bc3`) ALSO device-FALSIFIED** (2026-06-25, thinker containerized-KVM): removed `addNestedNavigation` entirely + hoisted the 4 bottom-nav graphs to top-level destinations in ONE Activity-hosted NavHost (official multi-back-stack pattern). Result — **C00/C01/C07/C08 PASS (zero nav regression)** but **C06 + C11 reproduce the IDENTICAL teardown ISE** on `route=search/search_input` (destination `0xe36e02dd`). Evidence `.lava-ci-evidence/lva008-cand7-gate/`. DEFINITIVE finding: the crash is **intrinsic to the `search_input` destination inside ANY graph, independent of nesting depth** — the `search` sub-graph entry collapses straight to DESTROYED, stranding the INITIALIZED `search_input` child. The cand7 branch is a clean no-regression refactor but provides ZERO LVA-008 benefit (DISCARD for the fix; may merge on architectural merits separately if desired). **ALL 8 app-level candidate classes now exhausted-and-device-falsified —**

CONCLUSIVELY an upstream androidx-navigation defect. **NEXT:** file the authored androidx minimal-repro + issue (`docs/issues/upstream/lva-008-androidx-navigation/`, branch `worktree-agent-a26086a188d3abfa6@71bee48c`, operator files it — needs a Google account); track for an upstream fix. No further app-level candidate remains. **Type:** Bug · **Severity:** P1 · **Workable item:** LVA-008

Symptom (CONFIRMED on device, 2026-06-08): the app PROCESS crashes at `MainActivity` destroy with `java.lang.IllegalStateException: State must be at least 'CREATED' to be moved to 'DESTROYED'` (top frame `androidx.lifecycle.LifecycleRegistryKt.checkLifecycleStateTransition`, `LifecycleRegistry.kt:92`), on the inner `search/search_input?query={query}&…` NavBackStackEntry. Reproduced by `Challenge11ArchiveOrgAnonymousSearchTest` (archive.org anonymous search) AND `Challenge06DownloadTorrentFileTest` (both deep-nav search → search_input → search_result → topic). Forensics: `.lava-ci-evidence/sixth-law-incidents/2026-06-08-navbackstackentry-teardown-crash-2.9.1-incomplete.json`.

Root cause (CONFIRMED): the bottom-nav graph is mounted as a NESTED `NavHost` from a parent `NavBackStackEntry` (`addNestedNavigation` in `app/.../navigation/MobileNavigation.kt`). The inner `NavController`'s host `LifecycleOwner` defaults to that parent entry. At activity-destroy the parent entry is driven straight to `DESTROYED`; the inner controller's host-lifecycle observer then walks its OWN back stack moving every entry to `DESTROYED` — including a `search_input` entry left `INITIALIZED` (created by the `popBackStack(); openSearchResult()` pattern + nested `saveState`/`restoreState`, never composed to `CREATED`). `LifecycleRegistry` rejects `INITIALIZED → DESTROYED`, killing the process.

Candidate-fix selection (per incident JSON ranking): prior device-FALSIFIED candidates — nav-compose `2.9.1 → 2.9.8`, `LenientTeardownRule` (uncatchable process death), move-search-to-outer-host, atomic `popUpTo` replace, and the `NavTeardownGuard` ON_STOP pruner (the stranded entry is NOT in the public `currentBackStack` StateFlow, so a public-API walk cannot reach it). The TOP-RANKED previously-UNTRIED candidate is the inner-NavHost lifecycle scoping (`next_hypotheses_untried[0]`: "Scope/lifecycle of the inner `rememberNestedNavigationController` vs the outer host at destroy"). That is what this fix implements.

Fix: bind the inner (nested) `NavHost`'s host `LifecycleOwner` to the Activity's view-tree `LifecycleOwner` instead of the parent `NavBackStackEntry`. The Activity reaches `CREATED` during its own normal lifecycle (driving inner entries to at least `CREATED` before any teardown) and its destroy path runs the inner entries through the regular `RESUMED → … → CREATED → DESTROYED` backward pass rather than the parent entry's abrupt collapse to `DESTROYED`. Only `LocalLifecycleOwner` is re-pointed — `LocalViewModelStoreOwner` +

`LocalSavedStateRegistryOwner` remain the parent entry, so nested-graph ViewModels still clear when the nested destination leaves the back stack (no leak / no scope regression).

Affected files:

- `core/navigation/src/main/kotlin/lava/navigation/ui/NavigationHost.kt` — new `activityScopedLifecycle: Boolean = false` param; when true, wraps the `NavHost` in `CompositionLocalProvider(LocalLifecycleOwner provides <view-tree owner>)`; new private `rememberActivityScopedLifecycleOwner()` using `LocalView.findViewTreeLifecycleOwner()` (same pattern already used in `core/ui/.../ModalBottomDialog.kt`).
- `core/navigation/src/main/kotlin/lava/navigation/ui/MobileNavigation.kt` — `NestedMobileNavigation` passes `activityScopedLifecycle = true` to its `NavigationHost` (the outer `MobileNavigation` host is unchanged → default `false`, preserving existing per-destination lifecycle scoping there).

Verification:

- **Compile (DONE, captured):** `./gradlew :core:navigation:compileDebugKotlin --max-workers=2 --no-daemon --offline` → `BUILD SUCCESSFUL in 37s` (only pre-existing deprecation/context-receiver warnings). The added defaulted param keeps all existing `NavigationHost` callers source-compatible.
- **Regression tests = the device Challenges (device-gate PENDING):** the load-bearing regression tests are the existing on-device `Challenge11ArchiveOrgAnonymousSearchTest` + `Challenge06DownloadTorrentFileTest`, which crash at teardown BEFORE this fix and must pass AFTER it. **No JVM/Robolectric regression test is feasible:** this is an androidx `LifecycleRegistry` teardown-ORDERING crash at real `Activity` destroy with a nested `NavHost` + a stranded `INITIALIZED` entry; the incident JSON proves it is even uncatchable by any in-process JUnit `TestRule` (it is process death inside `ActivityThread.performDestroyActivity`). Device-gate execution on this macOS host is BLOCKED by §6.AH-debt (container/VM emulator path does not yet boot here; no host-direct fallback permitted). The fix is therefore landed + compiled but **NOT yet device-verified** — it MUST be run on the Genymotion / emulator gate-host (C06 + C11) before this entry's status flips to "device-verified" and before any distribute. Prior sibling candidates were device-FALSIFIED, so this remains a hypothesis-with-strong-mechanism until the gate run, NOT a confirmed fix.

Falsifiability rehearsal (§6.N/§6.J, device-gate): a deliberate non-crashing break that the device Challenge would catch — revert `activityScopedLifecycle = true` to `false` in

`NestedMobileNavigation` (the production state this fix changes). Expected on-device result: C06 + C11 crash again at `MainActivity` destroy with the identical `IllegalStateException: State must be at least 'CREATED' to be moved to 'DESTROYED'` on the inner `search_input` entry (reverting reproduces the exact prior failure). Re-apply `true`; re-run; the teardown crash must not recur. (The rehearsal is device-gated for the same reason the fix is: the failure is a real-Activity teardown-ordering event with no JVM equivalent.)

Fix commit: on the agent worktree branch (see `git log` for SHA; this entry authored in the same commit as the fix per §6.T.4).
Forensic anchor / incident JSON (status updated): `.lava-ci-evidence/sixth-law-incidents/2026-06-08-navbackstackentry-teardown-crash-2.9.1-incomplete.json`

BUG-2026-06-25 — Search queries the WRONG provider set (video cluster #1/#2/#3/#5)

Reported: operator video, 2026-06-25 (`.lava-ci-evidence/video-analysis/`). Four symptoms, ONE causal cluster:

- **#2 (root):** user onboarded ONLY YTS, but Search queried RuTracker / RuTor / Internet Archive / Gutenberg (and result chips showed torrentdownloads / kinozal / yts) — search used the WRONG provider set, not the onboarded one.
- **#1:** those non-onboarded providers fail → zero results → full-screen "Something went wrong / Retry" (search appears completely broken).
- **#3:** the search-input provider chips disagreed with the results-screen provider chips, and the results chip set changed run-to-run for the same query.
- **#5:** no loading indicator + no empty-state — blank screen during/after search.

Root cause (FACT, file:line):

1. **`feature/search_input/.../SearchInputViewModel.kt:49-54` (pre-fix)** — a HARDCODED `availableProviders = [rutracker, rutor, archiveorg, gutenberg]`. The real onboarded providers live in `ProviderConfigRepository` (`provider_configs`, written by `OnboardingViewModel` `ensureDefault()`), and the live set is dynamic (vended by the lava-api-go `/providers` catalogue → `TrackerRegistry.populateFrom()`). `onCreate` (pre-fix `:84-85`) only flipped `selected` on chips that EXISTED in the hardcoded 4. A user who onboarded YTS (not in the 4) got ZERO selected chips, and the chip bar SET itself never showed YTS — it showed the 4 phantom providers.

2. **`SearchInputViewModel.kt:151` (pre-fix)** — `resolveProviderIdsForSubmit()` returned `null` when `selected.size == availableProviders.size` (the `null`-means-ALL sentinel). The empty-selection case produced `emptyList()`, which `SearchResultNavigation.kt:120` (`?.takeIf(isNotEmpty)`) DROPPED → deserialized at `:135-138` as `null` → `SearchResultViewModel.kt:100` `filter.providerIds == null` → `observePagingData()` (the single-tracker rutracker path) → 401/unauthorized → `SearchResultContent.Error` ("Something went wrong"). That is #1.

3. **#3 divergence + run-to-run instability** — input chips came from the hardcoded 4; result chips come from `state.filter.providerIds` (`SearchResultScreen.kt:238`). When the selection was dropped to `null`, result chips vanished entirely; and provider display names are filled by `LavaTrackerSdk.streamMultiSearch`'s `ProviderStart` events emitted from an UNORDERED parallel `channelFlow` (`LavaTrackerSdk.kt:806-826`), so names arrived non-deterministically.

4. **#5 blank screen** — `SearchResultScreen.kt:326-342` (pre-fix) rendered `items(filteredItems)` over the (empty) `Streaming.items` with NO loading indicator while providers were still SEARCHING → a blank screen during the in-flight search.

Fix (affected files):

- `feature/search_input/.../SearchInputViewModel.kt` — chip bar + query set are now built from `ProviderConfigRepository.observeAll()` (the source of truth), filtered by `searchEnabled && isEnabled`, **deterministically sorted by provider id** (kills #3 instability). Display names resolve from the live registry via the new `ProviderDisplayNameResolver` seam. `onCreate` populates chips via an Orbit `intent` (shared single intent queue → ordering guarantee). `resolveProviderIdsForSubmit()` ALWAYS returns the explicit selected list (the `null`-means-all sentinel is REMOVED; `null` only when nothing onboarded).
- `feature/search_input/.../ProviderDisplayNameResolver.kt` (new) + `SearchInputHiltModule.kt` (new) — narrow display-name boundary backed by `LavaTrackerSdk.listAvailableTrackers()`.
- `feature/search_input/build.gradle.kts` — adds `:core:tracker:client`.
- `feature/search_result/.../SearchResultScreen.kt` — Streaming branch now renders `loadingItem()` while any provider is still SEARCHING and no items have arrived (fixes #5 blank screen). The terminal Empty vs Error vs Content states are already produced by `SearchResultViewModel.handleStreamEnd()` (Empty on 0 results + no error; Error on 0 results + any provider ERROR).

Reproduce-first (RED) capture (§11.4.146 / §6.T.1): with the production VM mutated back toward the bug (re-introducing the `.filter { it in setOf("rutracker","rutor","archiveorg","gutenberg") }` intersection), the two video-scenario tests FAILED:

- `onCreate_with_onboarded_provider_outside_legacy_list_renders_it`
- `SubmitClick_with_provider_outside_legacy_list_queries_only_it`

`12 tests completed, 2 failed`. Mutation reverted → `BUILD SUCCESSFUL` (12/12).

Tests added (`feature/search_input/.../SearchInputViewModelTest.kt`):

- `onCreate_with_no_onboarded_providers_renders_empty_chip_bar`
- `onCreate_renders_chips_for_only_onboarded_search_enabled_providers`
- `onCreate_with_onboarded_provider_outside_legacy_list_renders_it` (YTS #2/#3)
- `SubmitClick_with_provider_outside_legacy_list_queries_only_it` (#2)
- `SubmitClick_providerIds_match_the_selected_chip_set_deterministically` (#3)
- rewrote 2 prior bluff tests that ENCODED the bug as expectation (`..._selects_no_chips` → `..._renders_empty_chip_bar`; `..._emits_null_providerIds` → `..._emits_explicit_list_not_null`) per §6.J.

Bluff-Audit: SearchInputViewModelTest Mutation: re-introduce the hardcoded-4 intersection in `onboardedSearchableProviders()` (the legacy bug). Observed-Failure: `expected:<[yts]> but was:<[]>` / `onCreate_with_onboarded_provider_outside_legacy_list_renders_it FAILED` + `SubmitClick_with_provider_outside_legacy_list_queries_only_it FAILED` (`12 tests completed, 2 failed`). Reverted: yes

GREEN: `:feature:search_input:testDebugUnitTest` + `:feature:search_result:testDebugUnitTest` both `BUILD SUCCESSFUL` (--max-workers=2 --no-daemon). Production Hilt/KSP for the new binding: `:feature:search_input:kspDebugKotlin` `BUILD SUCCESSFUL`.

§6.O-style note / OWED: the on-device confirmation is owed at the next (1076) §6.Z gate — the new search-provider Challenge (only-onboarded-provider search returns its results, chips agree, loading→empty/content) + C48–C52 must run on the Containers/VM emulator gate-host (§6.AH). The full `:app` build is env-gated here (`.env` + `keystores/debug.keystore` absent in the worktree per §6.H), so the device gate runs on the provisioned gate-host. The JVM RED→GREEN above is the unit-level proof.

Fix commit: on the agent worktree branch (authored in the same commit as the fix per §6.T.4).

2026-07-03 — `.torrent` download broken for non-rutracker providers (kinozal got an HTML error page, not the file)

Root cause (two chained defects):

1. **Cert / DownloadManager.** `DownloadTorrentUseCase` built a URI for the OS `DownloadManager`, which fetched the goapi `/download` URL over the SYSTEM trust store and rejected the self-signed goapi cert → the download never completed for the user (CertPathValidatorException on device).
2. **Provider-aware routing (#10).** The reroute (defect 1's fix) fetched the bytes in-app via `networkApiRepository.download` → `ProxyNetworkApi.download` → the goapi ROOT `/download/:id`, which is **rutracker-only** (`cmd/lava-api-go/main.go:140` wires `scraper = rutracker.NewClient`; `internal/handlers/torrent.go` `GetDownload` calls `h.scraper.GetTorrentFile` → rutracker.org). rutracker `.torrent`s worked, but kinozal (any non-rutracker) fetched from rutracker.org with a kinozal id/cookie → rutracker HTML error page → the bencode guard 502'd → DownloadState.Error → FAIL.

Affected files: `core/domain/.../DownloadTorrentUseCase.kt`, `core/downloads/.../DownloadServiceImpl.kt`, `core/downloads/api/DownloadRequest.kt`, `core/tracker/client/.../LavaTrackerSdk.kt`, `core/network/api/TorrentDownloadSource.kt` (new), `core/network/impl/.../TorrentDownloadSourceImpl.kt` (new), `core/network/impl/di/NetworkModule.kt`, `feature/topic/.../TopicViewModel.kt`.

Fix: (1) fetch the `.torrent` bytes IN-APP over the app's trusted OkHttp client (bencode-validate → write to public Downloads), removing the DownloadManager cert path. (2) route the in-app fetch through the provider-aware `/v1/{provider}/download/:id` SDK path (new `TorrentDownloadSource` mirroring `HttpDownloadSource`; `DownloadTorrentUseCase(trackerId, id, title)`; `TopicViewModel` threads `providerId`), so each provider's `.torrent` is fetched from ITS tracker with ITS session (both auth gates) — mirroring the device-green gutenberg HTTP-file download.

Verification: core+feature JVM 654 tests GREEN; kinozal 1080p `.torrent` keystone DEVICE-GREEN (`verdict=PASS`, "Download completed" ×15, C70-RESULT DOWNLOAD-OK) on the containerized emulator (API 34, goapi backend). Incident: `.lava-ci-evidence/sixth-law-incidents/2026-07-03-rutracker-kinozal-torrent-download-stall.json`.

Fix commit: this commit (Lava-Android-1.3.13-1079).